



UNIVERSITATEA DIN  
BUCUREȘTI

FACULTATEA DE  
MATEMATICĂ ȘI  
INFORMATICĂ



SPECIALIZAREA INFORMATICĂ

Lucrare de licență

# PROXIMAL POLICY OPTIMIZATION PENTRU SUPER MARIO BROS

Absolvent

Petrescu Alexandru-Antonio

Coordonator științific

Conf. dr. Păduraru Ciprian

București, iunie 2024

## Rezumat

În ultimele două decenii, s-au făcut nenumărate avansuri în domeniul de Deep Reinforcement Learning. O arie în care agenții de Deep RL au prins puternic avans este domeniul jocurilor video, în special al jocurilor create pentru consola de Atari 2600. Un exemplu remarcabil îl reprezintă Agent 57 creat de DeepMind care a reușit să rezolve o suită de 57 jocuri Atari 2600 cu o performanță remarcabilă și o putere de generalizare nemaîntâlnită. Cu toate acestea, puține progrese au fost făcute în domeniul jocurilor NES (Nintendo Entertainment System).

În această lucrare abordez antrenarea unui agent de Deep RL pe jocul Super Mario Bros prin folosirea algoritmului de învățare on-policy de Proximal Policy Optimization și combaterea problemei des întâlnite în domeniul ML de Catastrophic Forgetting prin folosirea metodei de Elastic Weight Consolidation. Rezultatele obținute ilustrează eficiența folosirii inputului de tip RAM față de cel de tip imagine, extinzând astfel posibilitățile de cercetare și studiere temeinică a altor jocuri NES.

## Abstract

In the last two decades, numerous advances have been made in the field of Deep Reinforcement Learning. One area where Deep RL agents have made significant progress is in video games, especially games created for the Atari 2600 console. A remarkable example is Agent 57, created by DeepMind, which managed to solve a suite of 57 Atari 2600 games with outstanding performance and an unfathomable power of generalization. However, little progress has been made in the field of NES (Nintendo Entertainment System) games. In this paper, I address the challenge of training a DRL agent to play the game of Super Mario Bros using the on-policy learning algorithm of Proximal Policy Optimization and combating the commonly encountered problem in the field of ML of Catastrophic Forgetting by using the Elastic Weight Consolidation method. The obtained results illustrate the efficiency of using RAM input compared to image input, thus expanding the possibilities for thorough research and study of other NES games.

# Cuprins

|          |                                            |           |
|----------|--------------------------------------------|-----------|
| <b>1</b> | <b>Introducere</b>                         | <b>5</b>  |
| 1.1      | Prezentare generală . . . . .              | 5         |
| 1.2      | Motivație . . . . .                        | 7         |
| <b>2</b> | <b>Lucrări asemănătoare</b>                | <b>9</b>  |
| 2.1      | Skip and Stack Wrapper . . . . .           | 9         |
| 2.2      | Trust region policy optimization . . . . . | 10        |
| 2.3      | Proximal policy optimization . . . . .     | 10        |
| 2.4      | Elastic Weight Consolidation . . . . .     | 11        |
| <b>3</b> | <b>Fundamente teoretice</b>                | <b>13</b> |
| 3.1      | Deep reinforcement learning . . . . .      | 13        |
| 3.1.1    | Agentul și mediul . . . . .                | 13        |
| 3.1.2    | Spațiul de stări și acțiuni . . . . .      | 14        |
| 3.1.3    | Procese Markov decizionale . . . . .       | 14        |
| 3.1.4    | Politici . . . . .                         | 15        |
| 3.1.5    | Tranziții și traiectorii . . . . .         | 16        |
| 3.1.6    | Recompensa și funcția de return . . . . .  | 16        |
| 3.1.7    | Politica optimă . . . . .                  | 17        |
| 3.1.8    | Funcțiile de valoare . . . . .             | 17        |
| 3.1.9    | Funcția avantaj . . . . .                  | 18        |
| 3.1.10   | Tipuri de algoritmi . . . . .              | 19        |
| 3.1.11   | Policy Gradient Optimization . . . . .     | 20        |
| <b>4</b> | <b>Metode și implementare</b>              | <b>22</b> |
| 4.1      | Structura proiectului . . . . .            | 22        |
| 4.2      | Regulile jocului . . . . .                 | 23        |
| 4.3      | Recompensa Agentului . . . . .             | 24        |
| 4.4      | Alegerea algoritmului . . . . .            | 25        |
| 4.5      | Construirea agentului . . . . .            | 26        |
| 4.5.1    | Mediul de Open AI Gym . . . . .            | 26        |

|          |                                                                   |           |
|----------|-------------------------------------------------------------------|-----------|
| 4.5.2    | Wrappere . . . . .                                                | 28        |
| 4.5.3    | Input tip imagine . . . . .                                       | 28        |
| 4.5.4    | Grayscale . . . . .                                               | 29        |
| 4.5.5    | Resizing . . . . .                                                | 30        |
| 4.5.6    | Frame Stacking . . . . .                                          | 30        |
| 4.5.7    | Medii Vectorizate . . . . .                                       | 31        |
| 4.5.8    | VecMonitor si VecNormalize . . . . .                              | 31        |
| 4.6      | Antrenarea agentului . . . . .                                    | 33        |
| 4.6.1    | Actorul si Criticul . . . . .                                     | 33        |
| 4.6.2    | Hyperparametrii . . . . .                                         | 33        |
| 4.7      | Input de tip RAM . . . . .                                        | 35        |
| 4.7.1    | Memoria RAM . . . . .                                             | 35        |
| 4.7.2    | Utils . . . . .                                                   | 36        |
| 4.7.3    | Frame Skipping si Frame Stacking . . . . .                        | 37        |
| 4.7.4    | Arhitectura Rețelei . . . . .                                     | 37        |
| 4.7.5    | Catastrophic Forgetting și Elastic Weight Consolidation . . . . . | 38        |
| <b>5</b> | <b>Evaluarea rezultatelor</b>                                     | <b>39</b> |
| 5.1      | Input de tip imagine . . . . .                                    | 39        |
| 5.1.1    | Rata de învățare constantă . . . . .                              | 39        |
| 5.1.2    | Rata de învățare liniară . . . . .                                | 42        |
| 5.2      | Input de tip RAM . . . . .                                        | 43        |
| 5.3      | Elastic Weight Consolidation . . . . .                            | 45        |
| <b>6</b> | <b>Concluzie</b>                                                  | <b>48</b> |
|          | <b>Bibliografie</b>                                               | <b>50</b> |

# Capitolul 1

## Introducere

### 1.1 Prezentare generală

Conceptul de o mașină autonomă care să nu aibă nevoie de intervenția umană pentru a funcționa a existat încă din antichitate. Cu toate acestea, teoria n-a fost dezvoltată până în 1950 când Alan Turing a inventat termenul de *inteligență artificială* în lucrarea sa intitulată *Computer Machinery and Intelligence*. De atunci, numeroase progrese, descoperiri și rezultate au fost obținute și spațiul de aplicabilitate a inteligenței artificiale s-a extins în nenumărate domenii, cum ar fi economia, industria automobilelor, a sănătății și a transportului.

Un model de inteligență artificială imită procesele naturale din creier în încercarea de a învăța corelațiile dintre inputurile și outputurile din seturile de date care îi sunt puse spre antrenare, cu scopul de a putea generaliza pe seturi de date pe care nu le-a mai văzut, folosindu-se astfel de experiențele acumulate. Întrucât sarcinile care pot să fie date către un program inteligent sunt variate și pot încorpora diferite restricții, tehnologiile folosite în crearea acestor modele s-au înmulțit și perfecționat în multe arii. Astfel avem domeniul de *machine learning* care este împărțit în *supervised learning* și *unsupervised learning*. Acestea, la rândul lor, pot fi clasificate mai departe în funcție de natura procesului de învățare și de adaptare la noi cerințe care pot să apară pe parcurs.

O ramură recent apărută și dezvoltată a inteligenței artificiale este *reinforcement learning* care se concentrează pe interacțiunea dintre un agent cu putere de generalizare și un mediu dinamic. Un agent de *supervised learning* primește niște date de antrenare pe care le folosește pentru a învăța corelațiile dintre input și output. Pe de altă parte, un agent de *reinforcement learning* primește, de fiecare dată când este rulat, alte date de intrare, întrucât mediul se schimbă în funcție de deciziile pe care agentul le ia, adăugând astfel un strat adițional de complexitate.

Tehnicile de *Reinforcement Learning* au fost folosite prima dată de către *Arthur Samuel* pentru a învăța un agent să joace dame. În deceniile următoare, metodele de RL au

fost perfecționate și rafinate, ducând la dezvoltarea unor algoritmi avansați precum Q-Learning, SARSA și, mai recent, metodele de Deep Reinforcement Learning (DRL), care combină RL cu rețelele neurale adânci. Aceste tehnici moderne au permis agenților să învețe comportamente complexe și să funcționeze la niveluri superioare în diverse domenii.

Un exemplu remarcabil de aplicare a DRL este modelul AlphaGo [1] dezvoltat de DeepMind, care a reușit să învingă campionul mondial la Go, un joc de strategie extrem de complex, demonstrând capacitatea acestor algoritmi de a depăși performanța umană în anumite sarcini. De asemenea, DRL a fost aplicat cu succes în robotică, unde agenții învață să manipuleze obiecte, să navigheze în medii necunoscute și să colaboreze cu alte mașini sau cu alți oameni.

În industria auto, DRL contribuie la dezvoltarea mașinilor autonome, optimizând deciziile de conducere în timp real pentru a îmbunătăți siguranța și eficiența acestora. În domeniul sănătății, RL este folosit pentru a crea planuri de tratament personalizate și pentru a optimiza fluxurile de lucru în spitale. În sectorul financiar, DRL ajută la dezvoltarea strategiilor de tranzacționare algoritmică și la gestionarea riscurilor.

De asemenea, Deep Reinforcement Learning a fost aplicat în mod eficient și în domeniul de gaming, în special în rezolvarea seriei de jocuri Atari 2600. Un agent de DRL numit *Agent 57*, creat de DeepMind, a reușit să învețe să joace 57 de jocuri de Atari 2600. Unul din motivele pentru care jocurile Atari au fost alese ca mediu propice de învățare este rezoluția redusă de 160x192 pixeli și memoria folosită de un cartuș Atari de 16KB. Pe lângă aceasta, funcția de reward poate fi ușor calculată prin diferența de scoruri de la un frame la altul, rezultând la o rată de convergență stabilă a modelului și un proces de învățare îmbunătățit și constant.



Figura 1.1: Suita de jocuri Atari 2600 folosită de Agent 57

Însă pentru celelalte domenii de jocuri video, cum ar fi NES (Nintendo Entertainment System), nu s-a cercetat mult prin metodele de Deep RL. Există mai multe motive pentru acest lucru, unul din ele fiind creșterea de rezoluție de la 160x192 pentru Atari, la 240x256 pentru NES și prezența unui set de acțiuni mărit. Acestea au adăugat la complexitatea rețelelor neurale adânci și a politicii ce ar trebui să fie învățate de către agent.

Scopul acestei lucrări este de a obține o înțelegere satisfăcătoare în ceea ce privește rezolvarea jocului Super Mario Bros de către AI. Spre deosebire de alte proiecte în care s-a încercat completarea acestui joc prin crearea a câte unui agent pentru fiecare nivel în parte, eu am aplicat și EWC (Elastic Weight Consolidation) pentru rezolvarea problemei de *Catastrophic Forgetting*. Acest algoritm intervine în majoritatea aplicațiilor de *machine learning* unde modelul preantrenat este pus să învețe pe alte task-uri fără să uite strategiile învățate anterior.

## 1.2 Motivație

Așa cum am spus mai sus, cea mai mare parte a efortului depus în aria antrenării agenților de Deep RL pentru jocurile video a fost concentrată în rezolvarea domeniului Atari, a jocurilor de strategie cum ar fi șah sau Go și a mediilor de simulare ce fac parte din librăria de Open AI Gym din Python. Însă pentru domeniul NES, fie sunt publicate foarte puține articole științifice, fie jocurile care au fost dezvoltate pentru consolă nu sunt studiate în profunzime. Două din motivele principale pentru această lipsă de evoluție sunt rezoluția mare de 240x256 pixeli cu cele 3 canale de RGB și numărul ridicat de acțiuni

posibile pe care agentul poate să le execute.

Dorind să-mi dezvolt abilitățile de RL, am hotărât să construiesc un agent de Deep RL pentru Super Mario Bros, un joc pentru NES care este adesea citat drept unul dintre cele mai bune și cele mai bine vândute jocuri din toate timpurile.

Voi extinde analiza făcută în acest domeniu prin utilizarea metodei de Elastic Weight Consolidation în încercarea de a antrena modelul să completeze sarcini succesive, fără a pierde performanța din sarcinile anterioare.



# Capitolul 2

## Lucrări asemănătoare

### 2.1 Skip and Stack Wrapper

Lucrarea de cercetare a lui David Gregory LeBlanc și Greg Lee [3] explorează aplicarea algoritmului de *Deep-Q Learning* pentru jocurile de Ping Pong de pe Atari dar și pentru jocurile de Arkanoid, Mega Man și Super Mario Bros.

În articol este studiată performanța algoritmului de Deep-Q Learning cu hyperparametri diferiți pentru numărul de frame-uri stivuite și sărite între oricare doua frame-uri stivuite. Așa cum se poate observa și din grafice, pentru jocul de Ping Pong, cei mai buni hyperparametri sunt 4 pentru numărul de frame-uri stivuite și 3 pentru numărul de frame-uri sărite.

Deși modelul ajunge la cel mai bun rezultat atins de cel mai bun jucător de Ping Pong, acela de 21, pentru jocul de Arkanoid de pe consola NES, agentul dă dovadă de dificultate în învățarea unei politici optime. Graficul de reward prezintă zgomot, factor al instabilității procesului de învățare de care dispune agentul timp de aproximativ 5 milioane de pași. Un motiv poate fi șablonul de albastru închis al jocului dar și scorul colorat cu roșu de lângă ecranul principal unde acțiunea are loc, acestea încurcând agentul din a distinge bila și platforma mișcătoare care e controlată de către agent.

În cazul jocului de Super Mario Bros, agentul dă iarăși dovadă de zgomot în graficul de reward, însă învătă cu succes să treacă peste obstacole și să ajungă la sfârșitul nivelului. Recompensa este concepută astfel încât agentul să fie răsplătit pentru cât de mult completează din nivel și penalizat pentru fiecare secundă care se scurge. Din păcate, această metodă prezintă semne de *reward farming*: agentul alege un obiectiv alternativ contrar scopului final al jocului, având ca rezultat doar maximizarea recompensei și nu completarea nivelului.

Această lucrare de cercetare m-a ajutat în implementarea unui wrapper de *Frame Skipping and Stacking* cu hyperparametri optimi pentru mediul de Super Mario Bros. Deși nu folosesc algoritmul de *Deep-Q Learning* ci metoda de PPO (Proximal Policy Optimi-

zation), abordarea folosită de către David Gregory LeBlanc și Greg Lee în explorarea tehnicilor prezentate și a diferitelor funcții de recompensă mi-au oferit o fundație solidă în încercarea mea de a construi un agent optim pentru jocul de Super Mario Bros.

## 2.2 Trust region policy optimization

Trust Region Policy Optimization (TRPO) [6] este un algoritm esențial în aria de reinforcement learning datorită capacității sale de a menține stabilitatea și performanța optimă a agenților. În contextul jocului de Super Mario Bros, algoritmul de TRPO oferă multiple avantaje în antrenarea agenților pentru a naviga niveluri complexe fără a uita informațiile învățate anterior.

Unul dintre cele mai mari avantaje ale algoritmului de TRPO este capacitatea sa de a asigura stabilitatea actualizărilor politicii agentului. TRPO utilizează o metodă de optimizare care limitează schimbările drastice în parametrii politicii, menținând variațiile într-o „regiune de încredere” (trust region). Acest lucru previne ca agentul să facă schimbări bruste care ar putea destabiliza procesul de învățare, menținând astfel performanțele consistente pe termen lung.

Algoritmul de TRPO asigură un echilibru optim între explorare și exploatare, permițând agentului să descopere noi strategii fără a renunța la tacticile eficiente deja învățate. Acest lucru este deosebit de important în Super Mario Bros, unde agentul trebuie să învețe să se adapteze la noi tipuri de inamici și obstacole în fiecare nivel. Prin menținerea acestui echilibru, algoritmul de TRPO ajută la maximizarea performanței generale a agentului.

Algoritmul de TRPO maximizează funcția de valoare prin optimizări controlate, asigurând că agentul atinge performanțe superioare pe toate nivelurile din joc. Aceasta înseamnă că agenții pot învăța să îmbunătățească continuu strategiile lor pe măsură ce avansează prin nivelurile de joc, menținând totodată abilitățile învățate din nivelurile anterioare.

## 2.3 Proximal policy optimization

Algoritmul de Proximal Policy Optimization este o metodă de ultimă oră de on-policy gradient optimization pentru antrenarea agenților de DRL (Deep Reinforcement Learning) [7] cu o performanță mărită în medii complexe și dinamice. Ea este construită pe rezultatele obținute de către precursorii ei, VGP (Vanilla Gradient Policy) și TRPO (Trust Region Policy Optimization) .

Algoritmul de PPO abordează problemele de instabilitate des întâlnite în metodele de on-policy. Algoritmii tradiționali de gradient, cum ar fi REINFORCE și VGP, pot suferi din cauza actualizărilor mari ale politicii, ceea ce duce la antrenamente instabile și

convergențe slabe. PPO atenuează această problemă prin introducerea unei funcții obiectiv surrogată de clipping, care limitează actualizările politicii într-o regiune de încredere. Această abordare asigură că fiecare actualizare nu este prea mare, menținând stabilitatea și performanța procesului de învățare. Stabilitatea oferită de PPO este esențială pentru antrenarea unui agent în mediul divers și dinamic al jocului Super Mario Bros, unde creșterea constantă a mean rewardului în timp este crucială.

Super Mario Bros prezintă un mediu provocator cu spații complexe de stări și acțiuni, necesitând ca agentul să ia decizii în timp real bazate pe observații parțiale și rezultate incerte. Robustetea algoritmului de PPO în gestionarea unor astfel de complexități îl face o alegere ideală. Capacitatea algoritmului de a echilibra explorarea și exploatarea, combinată cu abilitatea sa de a se adapta la noi situații fără a uita cunoștințele anterioare, asigură că agentul poate face față dificultăților întâlnite, reprezentate de mulțimea variată de inamici și platforme existente în joc.

## 2.4 Elastic Weight Consolidation

Elastic Weight Consolidation este o tehnică puternică în domeniul de machine learning ce se remarcă prin întărirea rețelelor neurale adânci, deosebit de utilă pentru a depăși provocarea prezentată de Catastrophic Forgetting [2]. Acest fenomen apare atunci când o rețea neurală uită informațiile învățate anterior în momentul în care se antrenează pe sarcini noi. În contextul antrenării agenților pentru jocuri precum Super Mario Bros, EWC se dovedește a fi o metodă utilă în a depăși această problemă.

Unul dintre beneficiile cheie ale algoritmului de EWC este capacitatea sa de a permite agenților să rețină cunoștințele de la nivelurile anterioare în timp ce învață pe niveluri noi. De exemplu, atunci când un agent de Super Mario Bros este antrenat pe alte nivele, cunoștințele dobândite în navigarea Nivelului 1 pot fi cruciale pentru completarea lor. Algoritmul de EWC realizează acest lucru prin identificarea și păstrarea greutăților importante pentru sarcinile învățate anterior, prevenind astfel suprascrierea acestor greutăți în timpul fazelor noi de învățare.

Procesul implică calcularea importanței fiecărui parametru din rețeaua neurală în raport cu sarcinile anterioare și aplicarea unui termen de regularizare pentru a asigura că acești parametri importanți nu se modifică semnificativ. Această regularizare ajută la menținerea performanței în sarcinile inițiale, permițând totodată rețelei să se adapteze la noi provocări.

Implementarea EWC în antrenarea agenților Super Mario Bros asigură că abilitățile și strategiile învățate în Nivelul 1, cum ar fi evitarea inamicilor sau optimizarea mișcării, nu se pierd atunci când agentul progresează la Nivelul 2. Această capacitate de învățare continuă este esențială pentru dezvoltarea de agenți performanți.

Mai mult decât atât, algoritmul de EWC este deosebit de relevant pentru mediile de

joc unde învățarea secvențială este necesară. În Super Mario Bros, fiecare nivel se bazează pe cele anterioare, necesitând ca agentul să integreze și să rafineze cunoștințele sale în mod progresiv. Algoritmul de EWC susține această învățare secvențială prin asigurarea că rețeaua neurală a agentului reține cunoștințele acumulate de-a lungul nivelurilor anterioare, rezultând la o învățare mai eficientă și mai eficace a nivelurilor ulterioare.

# Capitolul 3

## Fundamente teoretice

### 3.1 Deep reinforcement learning

#### 3.1.1 Agentul și mediul

Orice aplicație de Reinforcement Learning constă dintr-un agent sau mai mulți agenți care sunt plasați în cadrul unui mediu dinamic. În cazul nostru, ne interesează doar scenariul cu un singur agent. Acesta are acces de obicei doar la o submulțime din totalitatea informațiilor ce definesc starea curentă a mediului. Acestei submulțimi o să-i spunem ”observație”. În urma observației primite de către mediu, agentul ia o acțiune care în schimb modifică starea curentă a mediului. În urma schimbării de stare, agentul poate primi o recompensă dacă acțiunea luată a fost una favorabilă sau o penalizare în cazul în care acțiunea luată a fost mai puțin favorabilă. Agentul continuă acest ciclu până când își atinge obiectivul sau este obligat de către mediu să se oprească.

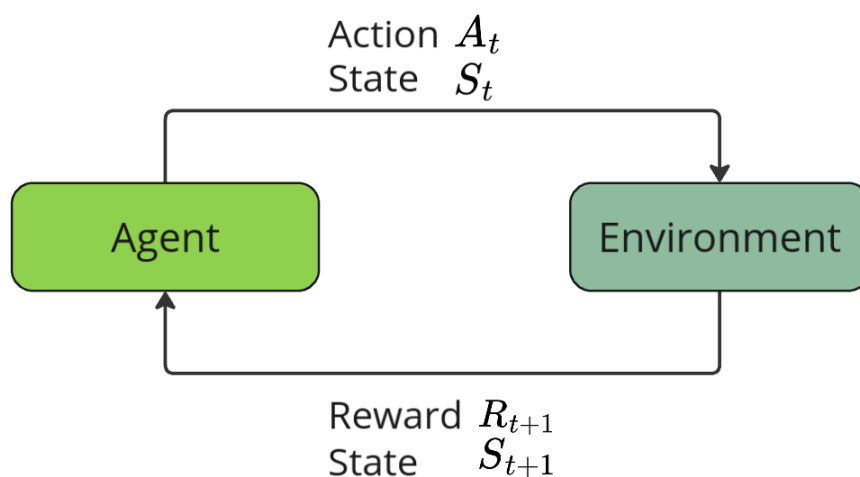


Figura 3.1: Interacțiunea dintre agent și mediu

### 3.1.2 Spațiul de stări și acțiuni

Spațiul de stări  $S$  reprezintă mulțimea tuturor stărilor posibile la care agentul are acces iar spațiul de acțiuni  $A$  reprezintă mulțimea tuturor acțiunilor pe care agentul poate să le ia în cadrul mediului. Spațiile de acțiuni se împart în două categorii: discrete și continue.

$$s \in S, a \in A$$

Există medii cum ar fi șah și Atari unde agentul poate să ia numai acțiuni discrete și sunt medii cum ar fi aplicațiile de robotica sau de condus al unui vehicul unde agentul poate lua doar acțiuni continue. Această diferență este foarte importantă întrucât există familii de algoritmi de Deep Reinforcement Learning care pot fi aplicate doar pe spații de acțiuni discrete și ar trebui modificate complet ca să meargă și cu celălalt tip de acțiuni.

### 3.1.3 Procese Markov decizionale

Toate problemele de RL pot să fie văzute prin prisma unui MDP (Markovian Decision Process). Într-un MDP, orice stare viitoare depinde doar de starea curentă. Cu alte cuvinte, viitorul și trecutul nu au relație cauzală.

$$\mathbb{P}[S_{t+1} \mid S_t] = \mathbb{P}[S_{t+1} \mid S_1, \dots, S_t]$$

Cu ajutorul acestei proprietăți, este mult mai ușor de definit soluția unei probleme de RL, întrucât tranzițiile dintre stări pot să fie vizualizate prin intermediul unui graf precum cel de mai jos:

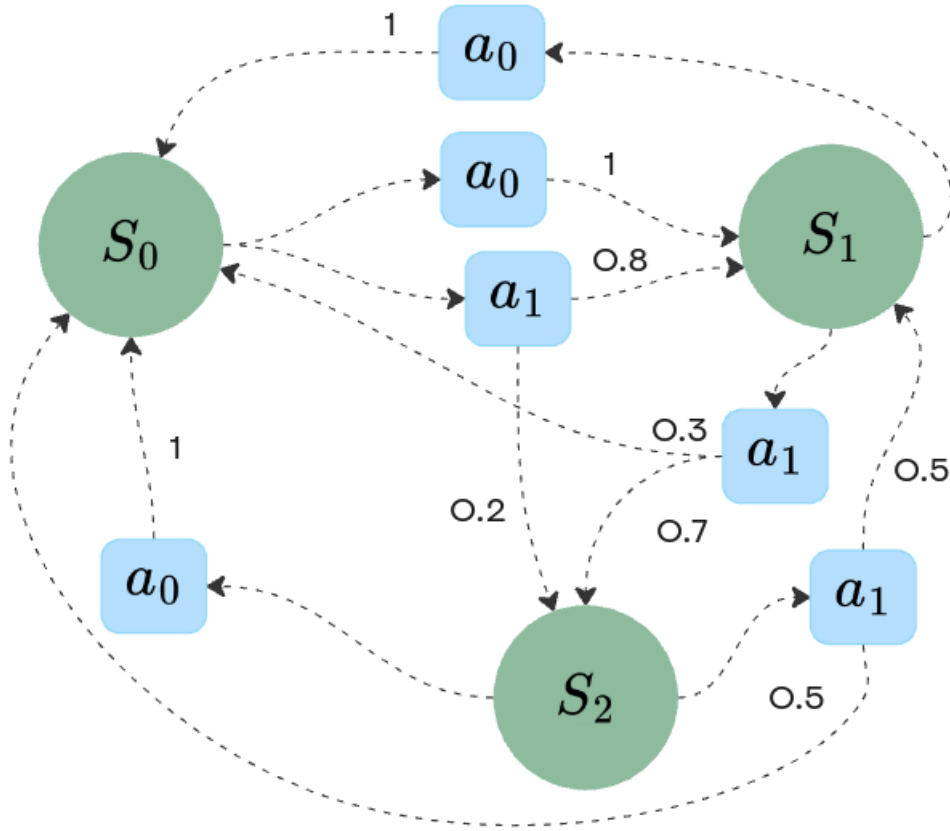


Figura 3.2: Un proces Markov decizional sub formă de graf

### 3.1.4 Politici

O politică reprezintă creierul agentului, cel care e responsabil pentru alegerea unei acțiuni fiindu-i dată o stare ca input. Politicile pot fi deterministe, în cazul în care agentul poate lua o singură acțiune aflându-se într-o stare, și stocastice, atunci când agentul poate alege din mai multe acțiuni la o stare curentă.

Pentru politicile deterministe se folosește următoarea notație:

$$a_t = \mu(s_t)$$

iar pentru cele stocastice se folosește notația:

$$a_t \sim \pi(\cdot|t)$$

În domeniul de Deep Reinforcement Learning folosim politici parametrizate al căror

proces de a lua acțiuni este reprezentat prin intermediul unei rețele neurale.

Greutățile și biasurile sunt reprezentate în general prin simbolurile de  $\phi$  și  $\theta$  pentru politicile deterministe, eventual stocastice:

$$a_t = \mu_\phi(s_t)$$

$$a_t \sim \pi_\theta(s_t)$$

### 3.1.5 Tranziții și traectorii

O traectorie este o serie  $\tau$  de stări și acțiuni consecutive luate de un agent, alături de recompensele pe care agentul le-a primit de-a lungul traseului:

$$\tau = (s_0, a_0, r_1, s_1, a_1, \dots, r_t, s_t)$$

Starea curentă este aleasă din *start-state distribution*:

$$s_0 \sim \rho_0(\cdot)$$

Tranzițiile între stări se fac pe baza acțiunilor luate de către agent și sunt definite de către mediul în care agentul se află. Întrucât lucrăm într-un MDP, următoarea stare  $s_{t+1}$  depinde doar de starea curentă  $s_t$  și acțiunea luată de către agent  $a_t$ . Tranzițiile pe care le vom folosi sunt stocastice.

$$s_{t+1} \sim P(\cdot | s_t, a_t) = \mathbb{P}(\cdot | S_t = s_t, A_t = a_t)$$

### 3.1.6 Recompensa și funcția de return

În urma tranziției dintr-o stare la alta, agentul primește o recompensă dată de către o funcție care este definită în felul următor:

$$r_t = R(s_t, a_t, s_{t+1})$$

Cu cât este mai mare recompensa, cu atât tranziția a fost mai favorabilă agentului. Mai sunt cazuri în care funcția de reward primește ca parametri starea curentă și acțiunea curentă  $r_t = R(s_t, a_t)$  sau doar starea curentă  $r_t = R(s_t)$ .

Scopul agentului este de a maximiza o funcție numită *return* peste o traectorie. Acest *return* poate fi de două feluri: *finite-horizon undiscounted return* și *infinite-horizon discounted return*.

Funcția de *finite-horizon undiscounted return* returnează suma tuturor recompenselor acumulate de către agent de-a lungul unei traectorii:



$$R(\tau) = \sum_{t=0}^T r_t$$

Funcția de *infinite-horizon discounted return* returnează suma tuturor recompenselor acumulate de-a lungul unei traiectorii după ce le înmulțește cu un discount  $\gamma \in (0, 1)$  ridicat la  $t$ :

$$R(\tau) = \sum_{t=0}^T \gamma^t r_t$$

Suma de *infinite-horizon discounted return* este preferată deoarece este mult mai ușor de lucrat cu ea în ecuații, întrucât poate converge către o limită. Pe lângă aceasta, este mult mai ușor de definit cerințe de *RL* folosind *infinite-horizon undiscounted return* deoarece recompensele ce sunt primite în prezent sunt mai importante decât cele primite în viitorul apropiat.

### 3.1.7 Politica optimă

Deși am vorbit despre cum agentul trebuie să maximizeze o funcție de *return* peste o traiectorie, de cele mai multe ori, politica este una stocastică. Din această cauză, ne interesează să maximizăm funcția de *return* în medie, cu alte cuvinte, să maximizăm *expected value* peste funcția de *return*.

Pentru a realiza acest lucru, trebuie să definim probabilitatea de a obține o traiectorie având deja o politică construită:

$$P(\tau|\pi) = \rho_0(s_0) \sum_{t=0}^{T-1} p(s_{t+1}|s_t, a_t) \pi(a_t|s_t)$$

Funcția de *expected return* este notată cu  $J(\pi)$  și este definită ca fiind:

$$J(\pi) = \int_{\tau} P(\tau|\pi) R(\tau) = \mathbb{E}_{\tau \sim \pi} [R(\tau)]$$

Prin urmare, politica optimă  $\pi^*$  pentru o cerință este cea care maximizează funcția  $J$ :

$$\pi^* = \underset{\pi}{\operatorname{argmax}} J(\pi)$$

### 3.1.8 Funcțiile de valoare

Este adesea util să cunoaștem valoarea unei stări sau acțiuni. Prin valoare, ne referim la rentabilitatea așteptată dacă începem în acea stare sau pereche de stare-acțiune și apoi acționăm conform unei anumite politici. Funcțiile de valoare sunt folosite în aproape fiecare algoritm DRL.

De cele mai multe ori ne interesează nu numai funcția de *expected return* ale unei politici, dar și să știm cât de performantă este o stare din care începem sau cât de avantajoasă este o acțiune în cadrul acelei politici. De aceea definim mai jos 4 funcții valoare pe care majoritatea algoritmilor de Deep RL le folosesc.

1. **On-Policy Value Function**  $V^\pi(s)$  este definită ca fiind funcția de *expected value* a funcției de return dacă începi în starea  $s$  și continui în funcție de politica  $\pi$

$$V^\pi(s) = E_{\tau \sim \pi} [R(\tau) | s_0 = s]$$

2. **On-Policy Action-Value Function**  $Q^\pi(s, a)$  este definită ca fiind funcția de *expected value* a funcției de return dacă începi în starea  $s$ , iei acțiunea  $a$  și continui să te ghidezi mai departe după politica  $\pi$

$$Q^\pi(s, a) = E_{\tau \sim \pi} [R(\tau) | s_0 = s, a_0 = a]$$

3. **Optimal Value Function**  $V^*(s)$  este definită ca fiind funcția de *expected value* a funcției de return dacă începi în starea  $s$  și continui conform politicii optime

$$V^*(s) = \max_{\pi} E_{\tau \sim \pi} [R(\tau) | s_0 = s]$$

4. **Optimal Action-Value Function**  $Q^*(s, a)$  este definită ca fiind funcția de *expected value* a funcției de return dacă începi în starea  $s$ , iei acțiunea  $a$  și continui să iei acțiuni conform politicii optime

$$Q^*(s, a) = \max_{\pi} E_{\tau \sim \pi} [R(\tau) | s_0 = s, a_0 = a]$$

### 3.1.9 Funcția avantaj

O funcție care ne mai ajută foarte mult în antrenarea unei rețele neurale în cadrul unui mediu dinamic este funcția avantaj. De cele mai multe ori nu ne interesează care este acțiunea optimă plecând dintr-o stare  $s$ , ci ne interesează cu cât mai avantajoasă este o acțiune  $a$  față de restul acțiunilor în general.

De aceea avem funcția avantaj  $A^\pi(s, a)$  care descrie acest lucru luând în considerare că după ce agentul alege acțiunea  $a$  plecând din starea  $s$ , el se ghidează mai departe conform politicii  $\pi$

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$$

### 3.1.10 Tipuri de algoritmi

În încercarea de a rezolva multitudinea de probleme RL ce au apărut de-a lungul timpului, mulți algoritmi au fost creați și perfecționați. Acești algoritmi pot să fie împărțiți în funcție de două criterii: accesul la mediu și modul în care este utilizată ultima politică. Astfel, avem următoarele categorii de algoritmi:

- **Model-based:** Agentul se bazează pe modelul mediului în care se află pentru a calcula politica optimă folosind programarea dinamică. Modelul poate să fie dat agentului sau să fie aflat de către acesta. De obicei, acești algoritmi nu sunt preferați pentru rezolvarea celor mai multe probleme din cauza instabilității procesului de învățare.
- **Model-free:** Agentul nu are nevoie de un model pentru a afla corelațiile dintre stări și a funcției de *expected return*. Deși sunt *sample inefficient*, acești algoritmi sunt folosiți în majoritatea problemelor de tip DRL.
- **On-Policy:** Agentul se folosește de ultima politică construită pentru a ajunge la următoarea politică. Unul dintre cei mai folosiți algoritmi de on-policy este *PPO* (Proximal Policy Optimization) și pe acesta l-am utilizat pentru rezolvarea jocului de Super Mario Bros.
- **Off-Policy:** Agentul își calculează actualizările de politică folosindu-se de oricare dintre politicile din trecut. Acești algoritmi încearcă să maximizeze o aproximare  $Q_\phi(s, a)$  a funcției de **optimal action-value function**  $Q^*(s, a)$ .

În secțiunea următoare voi vorbi doar despre cum algoritmii de *on-policy* funcționează, pentru a clarifica modul în care algoritmul de PPO lucrează în spate și ce încearcă el să maximizeze.

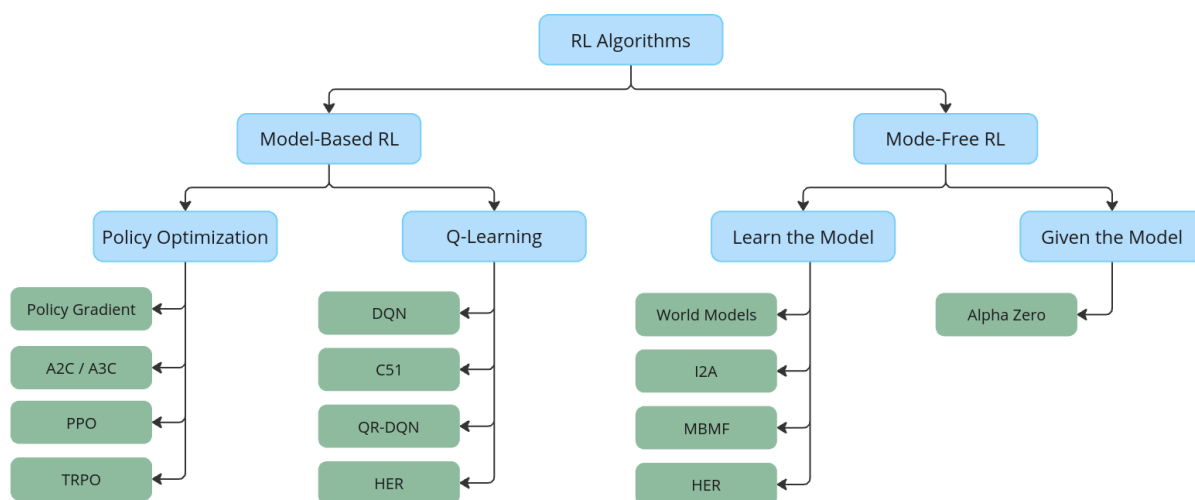


Figura 3.3: Ierarhie incompletă a algoritmilor de Deep RL

### 3.1.11 Policy Gradient Optimization

Scopul final al unui algoritm de *on-policy* este de a folosi o politică stocastică parametrizată  $\pi_\theta$  pentru a maximiza funcția de expected return  $J(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)]$ . În cazul acesta, funcția de return este cea de *finite-horizon undiscounted return* pentru simplitate, dar derivarea de mai jos ar trebui să fie identică și pentru celălalt tip de return.

Pentru a maximiza obiectivul algoritmului de *on-policy*, trebuie să folosim tehnica de *gradient ascent* pentru a găsi următoarea politică:

$$\theta_{k+1} = \theta_k + \alpha \nabla_\theta J(\pi_\theta) \big|_{\theta_k}$$

unde  $\nabla_\theta J(\pi_\theta)$  se numește gradient-ul politicii. Însă ne trebuie o aproximare a gradient-ului politicii care poate fi ușor calculată din traiectoriile pe care le acumulăm folosind ultima politica aflată. Fără să intrăm în detalii, folosind formula pentru logaritmul probabilității traiectoriei și trucul pentru logaritmul derivatei, estimarea căutată este următoarea:

$$\therefore \nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[ \sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t | s_t) R(\tau) \right]$$

Întrucât aproximarea gradient-ului de policy folosește un *expected value*, putem estima această valoare acumulând un număr  $n$  de traiectorii  $\tau_i$  cu  $i = 0, \dots, n-1$  folosind ultima politică  $\theta$ . Traiectoriile obținute sunt ulterior adunate și împărțite la  $n$  pentru a afla media lor.  $T$  reprezintă lungimea unei traiectorii.

$$\hat{g} = \frac{1}{n} \sum_{i=0}^{n-1} \sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t | s_t) R(\tau_i),$$

Estimarea de mai sus poate să fie îmbunătățită folosind tehnicile de *temporal structure* și *baselines*. O problemă pe care formula descrisă mai sus o are este faptul că, pentru fiecare acțiune  $a_i$  luată în cadrul unei traiectorii  $\tau_i$ , ne interesează reward-ul acumulativ pe care îl obținem după ce efectuăm acea acțiune și nu tot episodul în care ne aflăm. Astfel, se poate demonstra matematic că formula rezultată este asemănătoare cu cea scrisă mai sus:

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[ \sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t | s_t) \sum_{t'=t}^T R(s_{t'}, a_{t'}, s_{t'+1}) \right].$$

Un *baseline* este un termen ce poate fi aplicat sumei de recompense din structura temporală de mai sus fără a schimba rezultatul final, cu scopul de a reduce varianța preluată din datele colectate de către agent folosind ultima politică. După ce am aplicat acest termen, formula finală este:

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[ \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t \mid s_t) \left( \sum_{t'=t}^T R(s_{t'}, a_{t'}, s_{t'+1}) - b(s_t) \right) \right].$$

Astfel am obținut expresia matematică generală pentru optimizarea gradientului de politică. Termenul de *baseline* poate să aibă diverse forme, dar pentru o variantă cât mai mică, acesta poate fi înlocuit cu funcția de advantage  $A^{\pi}(s_t, a_t)$  descrisă într-o secțiune anterioară.

# Capitolul 4

## Metode și implementare

### 4.1 Structura proiectului

Pentru completarea jocului Super Mario Bros, am creat un agent de Deep Reinforcement Learning cu ajutorul algoritmului PPO. Codul acestui algoritm este preluat din librăria de Python **stable-baselines3** care este construită peste framework-ul PyTorch.

Structura proiectului constă în 3 foldere pentru jurnale, înregistrări și modele antrenate, 4 fișiere Jupyter notebook și 2 script-uri Python cu codul necesar rulării agenților. Două Jupyter notebooks sunt dedicate construirii, antrenării și testării de agenți folosind input-uri de tip imagine iar alte două fișiere sunt scrise pentru input-uri de tip RAM.

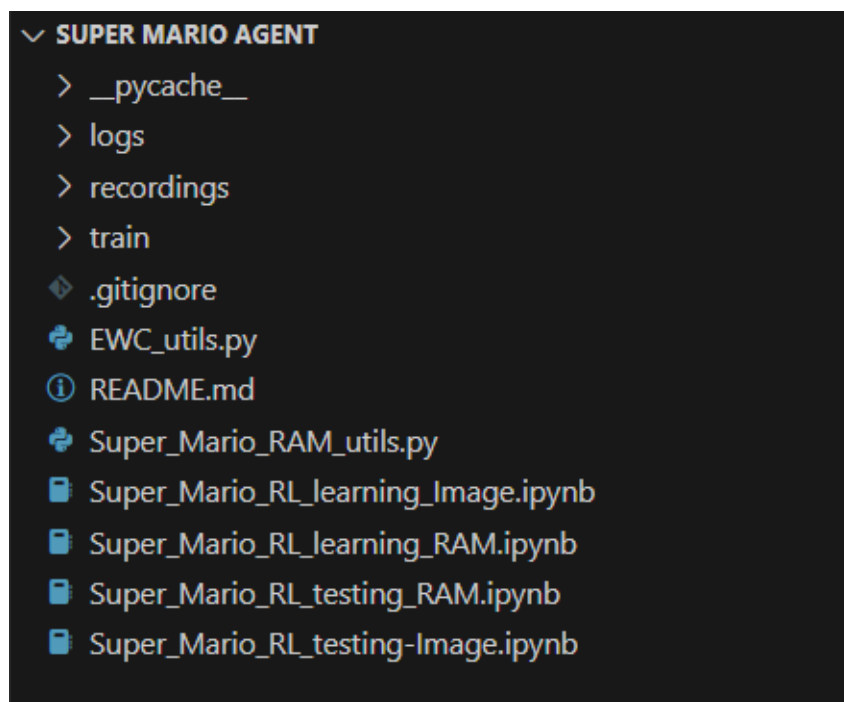


Figura 4.1: Structura proiectului

## 4.2 Regulile jocului

Jocul clasic de Super Mario Bros este un platformer în care jucătorul trebuie să depășească diferite obstacole, precum inamici și ziduri, pentru a ajunge la sfârșitul nivelului în care acesta se află. Jocul are în total 8 lumi, fiecare cu câte 4 nivele.

Jucătorul începe nivelul cu o singură viață. Pierderea acesteia are loc la atingerea unui inamic sau când acesta cade într-o groapă și acest lucru presupune reluarea nivelului.

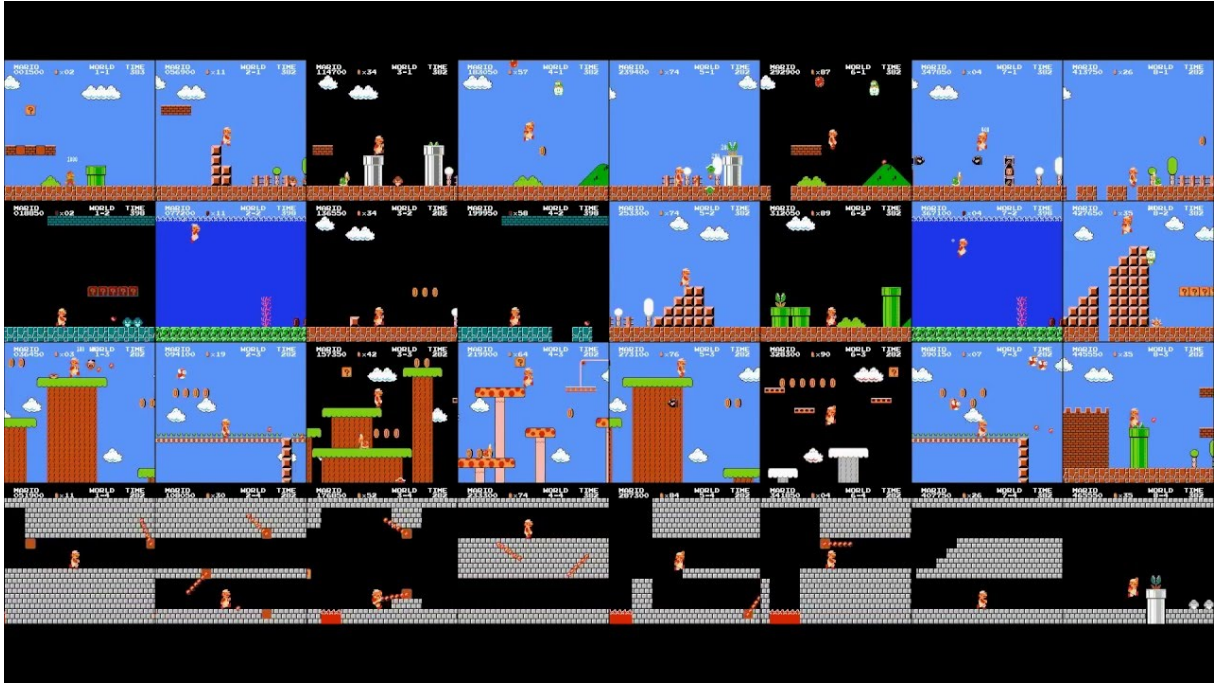


Figura 4.2: Toate nivelele din Super Mario Bros

Scopul agentului este de a învăța să depășească obstacolele din primul nivel într-un timp cât mai scurt. Pentru urmatorul nivel, agentul își folosește abilitățile învățate din primul nivel.

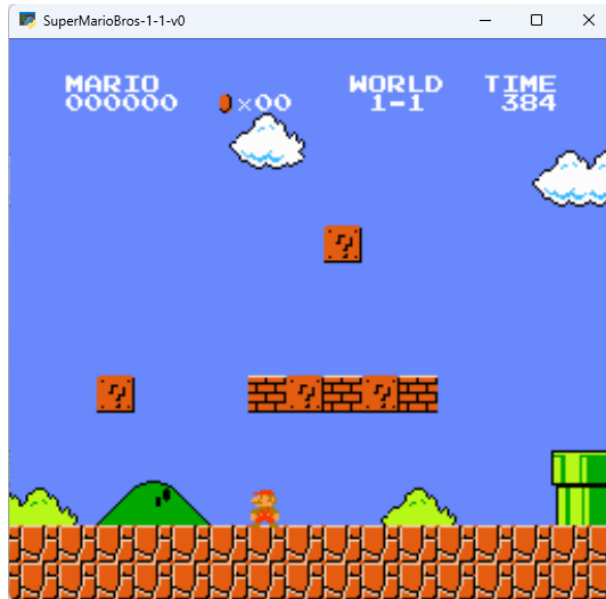


Figura 4.3: Agentul joacă primul nivel



Figura 4.4: Agentul joacă al doilea nivel

### 4.3 Recompensa Agentului

Agentul pornește nivelul cu un timp limitat. El este recompensat cu puncte proporțional cu distanța parcursă spre dreapta și penalizat pentru fiecare unitate de timp care trece în joc. Recompensa a fost construită pentru a motiva agentul să termine nivelul în care se află cât mai repede posibil.



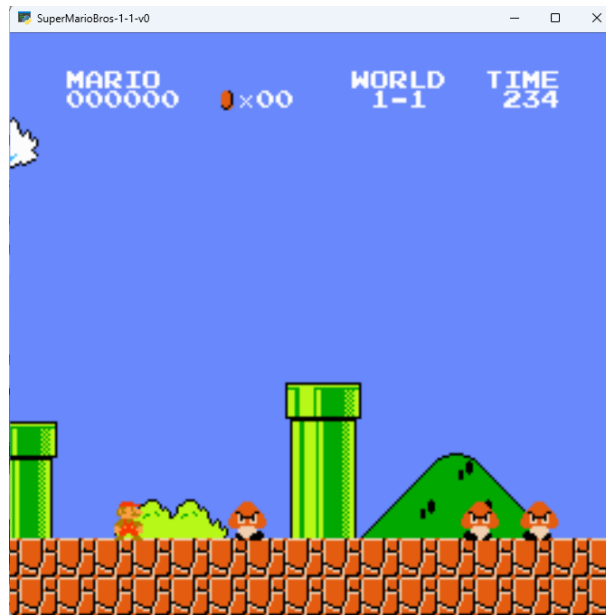


Figura 4.5: Goombas in primul nivel



Figura 4.6: Alte pericole in al doilea nivel

## 4.4 Alegerea algoritmului

Se folosește algoritmul de PPO pentru crearea modelului de DRL. Algoritmul ales este o variantă îmbunătățită a algoritmului de TRPO întrucât acesta calculează constrângerea de *KL-divergence* prin folosirea unei funcții specializate de clipping în funcția de obiectiv. Rezultatul produs este același ca cel obținut de către algoritmul de TRPO, acela fiind de a descuraja agentul din a se îndepărta prea mult de vechea politică.

De asemenea, algoritmul de PPO este considerat a fi o metodă de *on-policy optimization* de ultimă oră care a fost folosită pentru a învăța un model multi-agent să joace Dota 2 [4]. Deși algoritmul de PPO este sample inefficient deoarece el se bazează doar pe experiența acumulată folosind ultima politică creată, acesta este mult mai stabil decât metodele de Q-learning. Chiar mai mult, a fost demonstrat că metoda de PPO mereu converge către un optimum local folosind pași cât mai largi care să nu depășească constrângerea de *KL-divergence* ce există în cazul algoritmului de TRPO.

## 4.5 Construirea agentului

### 4.5.1 Mediul de Open AI Gym

Am folosit un mediu special de Open AI Gym pentru rularea jocului Super Mario Bros pe un emulator de NES (Nintendo Entertainment Systems) numit **gym-super-mario-bros**. Acest mediu are la dispoziție toate cele 32 de nivele ale jocului, valabile în 4 versiuni. Toate cele 4 versiuni au ca și spațiu de observații instanțe de **gym.spaces.Box**. Acestea prezintă valori de tipul *uint8* de la 0 până la 255 pentru dimensiunea de  $(240, 256, 3)$ , unde primele două dimensiuni reprezintă înălțimea și lungimea frame-ului iar ultima dimensiune reprezintă cele 3 canale de *RGB* ale imaginii. Din moment ce calitatea imaginii scade pe măsură ce creștem versiunea mediului, am antrenat toți agenții pe medii a căror versiune este zero pentru o ilustrare mai clară a jocului.

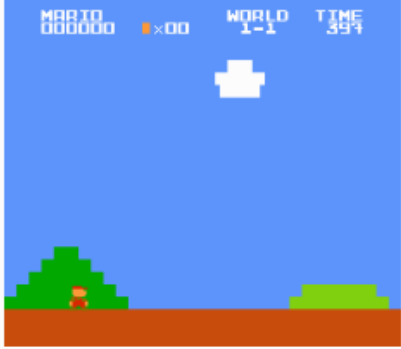
| Environment       | Game | ROM        | Screenshot                                                                                                                                                                                                                                                                                                                                |
|-------------------|------|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SuperMarioBros-v0 | SMB  | standard   |  A screenshot of the original Super Mario Bros. game. It shows Mario on a brick floor, a green hill, and a blue sky with a single cloud. The HUD at the top shows 'MARIO 000000', 'x00', 'WORLD 1-1', and 'TIME 397'.                                   |
| SuperMarioBros-v1 | SMB  | downsample |  A screenshot of Super Mario Bros. where the background is black, representing a downsampled or night-like version. Mario is on a brick floor, and the green hill is visible. The HUD is identical to the standard version.                            |
| SuperMarioBros-v2 | SMB  | pixel      |  A screenshot of Super Mario Bros. with a pixelated or low-resolution aesthetic. The background is blue with a single white pixelated cloud. Mario is on a brown ground, and the green hill is visible. The HUD is identical to the standard version. |
| SuperMarioBros-v3 | SMB  | rectangle  |  A screenshot of Super Mario Bros. with a rectangular or blocky aesthetic. The background is blue with several white rectangular clouds. Mario is on a brown ground, and the green hill is visible. The HUD is identical to the standard version.     |

Figura 4.7: Versiunile jocului de Super Mario Bros

### 4.5.2 Wrappere

Întrucât spațiul de acțiuni al mediului clasic de **gym-super-mario-bros** conține toate cele 256 de acțiuni discrete care sunt valabile pe sistemul de NES, librăria ne pune la dispoziție un wrapper care limitează spațiul de acțiuni la care agentul are acces. Wrapperul de **nes\_py.wrappers.JoypadSpace** primește ca inputuri mediul curent și o listă de acțiuni la care o să aibă acces agentul, unde librăria de **nes\_py** este folosită pentru controlarea emulatorului de NES. Aceste liste de acțiuni pot fi importate din **gym\_super\_mario\_bros.actions** și dau acces la acțiunile de tip *RIGHT\_ONLY*, *SIMPLE\_MOVEMENT* și *COMPLEX\_MOVEMENT*.

Pentru că ne dorim ca agentul nostru să parcurgă nivelele cât mai repede posibil, fără să descopere alte căi lăaturalnice care necesită mișcări mai complexe, am folosit lista de acțiuni de *SIMPLE\_MOVEMENT* unde agentul poate să meargă în dreapta, să sară, să stea pe loc sau să meargă spre stânga.

```
# actions for very simple movement
SIMPLE_MOVEMENT = [
    ['NOOP'],
    ['right'],
    ['right', 'A'],
    ['right', 'B'],
    ['right', 'A', 'B'],
    ['A'],
    ['left'],
]
```

Listing 4.1: Simple Movement Actions

### 4.5.3 Input tip imagine

Experimentele s-au realizat cu două tipuri de inputuri pentru agentul de Deep Reinforcement Learning. Câte două *Jupyter Notebook-uri* au fost create pentru folosirea inputului de tip imagine și cele de tip memorie RAM.

Mediul de Open AI Gym folosit pentru jocul de *Super Mario Bros* returnează frame-uri de dimensiunea (240, 256, 3). Pentru a antrena agentul într-o manieră eficientă și rapidă, dimensiunea inputului a fost redusă folosind tehnici clasice de preprocesare a mediului precum *grayscale*, *resizing* și *frame stacking*. Acestea sunt aplicate sub forma unor *Observation Wrappere* ce sunt importate din modulele de **gym.wrappers** și **stable\_baselines3.common.vec\_env**

```

# Changes the game from colour image (RGB) to grayscale so that our processing
  becomes faster as we need to process less data
from gym.wrappers import GrayScaleObservation, ResizeObservation

# VecFrameStack allows us to work with our stacked environments by letting us know
  the information of previous frames
from stable_baselines3.common.vec_env import VecFrameStack, SubprocVecEnv,
  VecMonitor, VecNormalize

```

Listing 4.2: Wrapperele aplicate asupra mediului

#### 4.5.4 Grayscale

Tehnica de grayscale reduce numărul de canale ale imaginii de la 3 canale la unul singur, îmbunătățind astfel memoria cu un factor de 3. Astfel, după aplicarea acestui wrapper, spațiul de observații o să ajungă la dimensiunea de  $(240, 256, 1)$ . Acesta folosește în spate funcția de `cv2.cvtColor`.

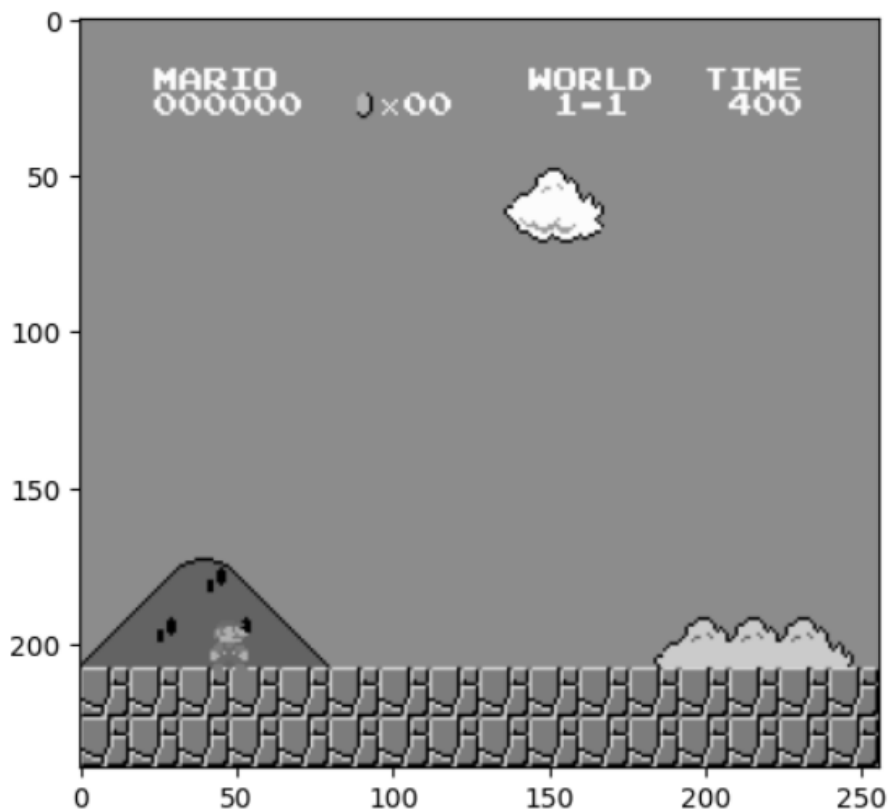


Figura 4.8: Frame dupa grayscale

### 4.5.5 Resizing

S-a folosit tehnica de resizing pentru a reduce dimensiunea frame-ului de la  $(240, 256, 1)$  la  $(84, 84, 1)$ .

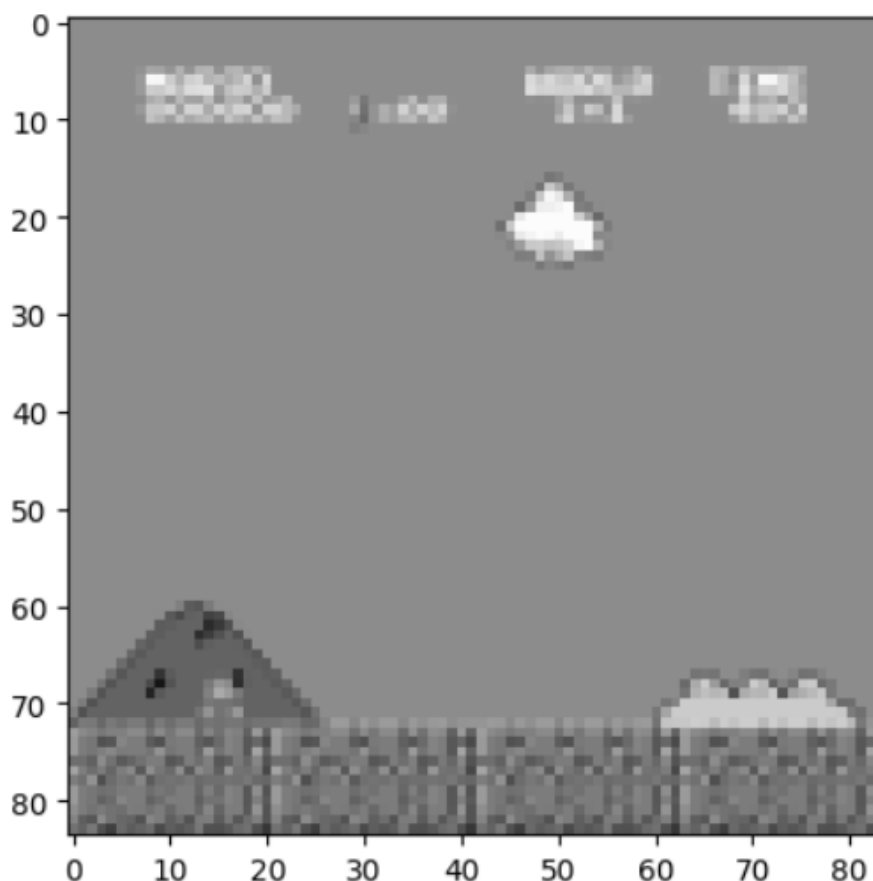


Figura 4.9: Frame dupa resizing

### 4.5.6 Frame Stacking

După aplicarea tehnicilor de resizing si grayscale, pentru a reda agentului noțiunea de mișcare într-o manieră cât mai compactă și eficientă, agentul a fost wrappuit într-un *Frame Stacking* wrapper folosind clasa de **VecFrameStack**. Acest wrapper stivuiește ultimele 4 frame-uri ale episodului într-un singur input, schimbând spațiul de observații de la  $(84, 84, 1)$  la  $(84, 84, 4)$ . Deși nu s-a testat cu numere diferite de cadre, s-a observat la *yumouwei* [8] care a testat cu câte 1, 2 și 4 frame-uri stivuite că agentul cel mai bine învață când îi pui la dispoziție ultimele 4 cadre întrucât este în stare să diferențieze dintre momentele când Mario se află în repaus și când Mario se află în mișcare în contextul evitării obstacolelor. De asemenea, în majoritatea cazurilor când un agent de DRL trebuie să învețe să joace un joc video, fie că e Atari, NES sau altă consolă de jocuri video relativ simplistă, tehnica de frame stacking este folosită cel mai des cu câte 4 frame-uri.

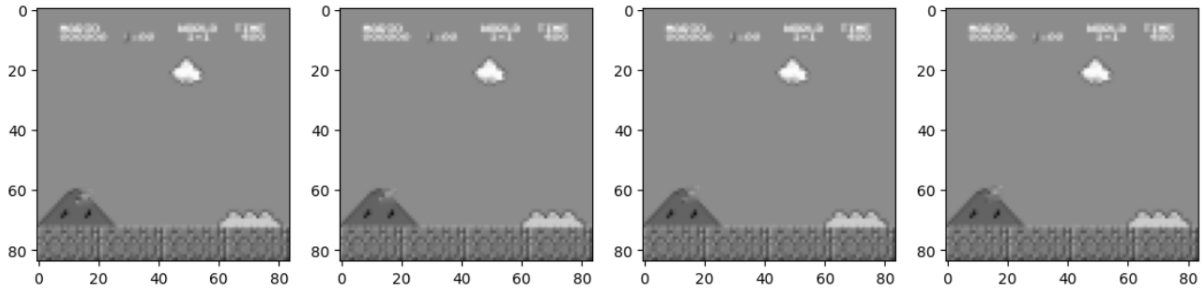


Figura 4.10: Ultimele 4 frame-uri din joc

### 4.5.7 Medii Vectorizate

În **stable-baselines3** există două tipuri principale de medii vectorizate care sunt folosite pentru antrenarea eficientă a agenților de *Deep RL*: **DummyVecEnv** și **SubprocVecEnv** din modulul de **stable\_baselines3.common.vec\_env**. **DummyVecEnv** este folosit pentru a crea unul sau mai multe medii care să ruleze secvențial pe același proces. De obicei, acest wrapper este folosit pentru medii simple sau în cazul în care dorești să lucrezi cu un singur mediu.

Mediul vectorizat **SubprocVecEnv** se diferențiază de **DummyVecEnv** prin faptul că acesta memorează și execută fiecare mediu pe câte un proces separat. Din această cauză, nu poți să lucrezi cu un număr de medii mai mare decât numărul de nuclee al procesorului sistemului de calcul pe care antrenezi agentul. De asemenea, memoria RAM folosită de către calculator este direct proporțională cu numărul de procese alocate wrapperului de **SubprocVecEnv**. Un avantaj pe care îl oferă **SubprocVecEnv** față de **DummyVecEnv** este viteza mărită pe care agentul o are în timpul antrenării din cauza multiprocessing.

S-a folosit wrapperul de **SubprocVecEnv** cu 6 medii generate, pentru a evita ca agentul să rămână fără memorie RAM de-a lungul procesului de învățare.

### 4.5.8 VecMonitor și VecNormalize

Pentru accesul timpului și reward-ului mediu al unui episod, s-a aplicat wrapperul de **VecMonitor** care păstrează toate aceste informații de-a lungul procesului de antrenare într-un fișier numit *monitor.csv*. Acest wrapper este folosit peste un wrapper de **DummyVecEnv** sau **SubprocVecEnv**, fiind varianta vectorizată a wrapperului de **Monitor**.

```

{"t_start": 1717777552.883347, "env_id": null}
r,l,t
107.0, 116, 51.92692
91.0, 136, 52.25457
88.0, 144, 52.393979
85.0, 150, 52.496226
97.0, 126, 54.1123
97.0, 132, 54.542929
791.0, 540, 59.299986
597.0, 468, 60.595668
97.0, 128, 62.777493

```

Listing 4.3: Sample Data Output

**VecNormalize** este un alt wrapper ce este aplicat asupra wrapper-ului de **VecMonitor** pentru a normaliza observațiile și reward-urile agentului. Normalizarea este o tehnică de îmbunătățire a stabilității agentului în timpul antrenării și îl ajută să convergă mai repede către un maxim local. Din cauza modului în care acest wrapper funcționează, s-a aplicat wrapperul de **VecMonitor** înainte aplicării wrapperului de **VecNormalize** pentru a nu avea metricele de *mean reward* și *mean length* din fișierul de *monitor.csv* modificate de către normalizare.

```

def make_env(seed, env_name):
    def init():
        env = gym_super_mario_bros.make(env_name)
        env = JoypadSpace(env, SIMPLE_MOVEMENT)
        env = GrayScaleObservation(env, keep_dim=True)
        env = ResizeObservation(env, (84, 84))
        env.seed(seed)
        return env
    return init

```

Listing 4.4: Funcția **make\_env** de creare a unui mediu de Super Mario Bros

```

env = SubprocVecEnv([make_env(i) for i in range(6)])
env = VecMonitor(env, "./opt_logs")
env = VecNormalize(env, clip_obs=255, clip_reward=15)
env = VecFrameStack(env, 4, channels_order="last")

```

Listing 4.5: Wrapperele vectorizate aplicate asupra mediului clasic



## 4.6 Antrenarea agentului

### 4.6.1 Actorul si Criticul

A fost folosită arhitectura de default a rețelei neurale convoluționale a algoritmului de PPO din implementarea de **stable-baselines3**. Rețeaua neurală este formată din părțile de CNN si MLP [5].

Partea de CNN se mai numeste si *features extractor* si aceasta este sharuita de actor si critic. Ea este formata din 3 layere convoluționale alaturi de functiile de activare *ReLU*, urmate de un ultim layer de *Flatten* pentru a pregati datele poentru rețeaua de MLP.

Rețeaua de MLP conține o bucată de layere sharuita de catre actor si critic si cate o parte specifica fiecaruia dintre acestea. Partea comuna este reprezentata de un singur layer linear urmat de o functie de activare *ReLU*. Ultima parte a rețelei de MLP difera de la actor la critic intrucat actorul trebuie sa primeasca o lista de *logit-uri* ce reprezinta (dupa aplicarea algoritmului de Softmax) probabilitatea fiecărei actiuni de a fi aleasa de catre agent iar criticul returneaza un tensor format dintr-un singur numar ce reprezinta estimarea functiei de value a starii curente.

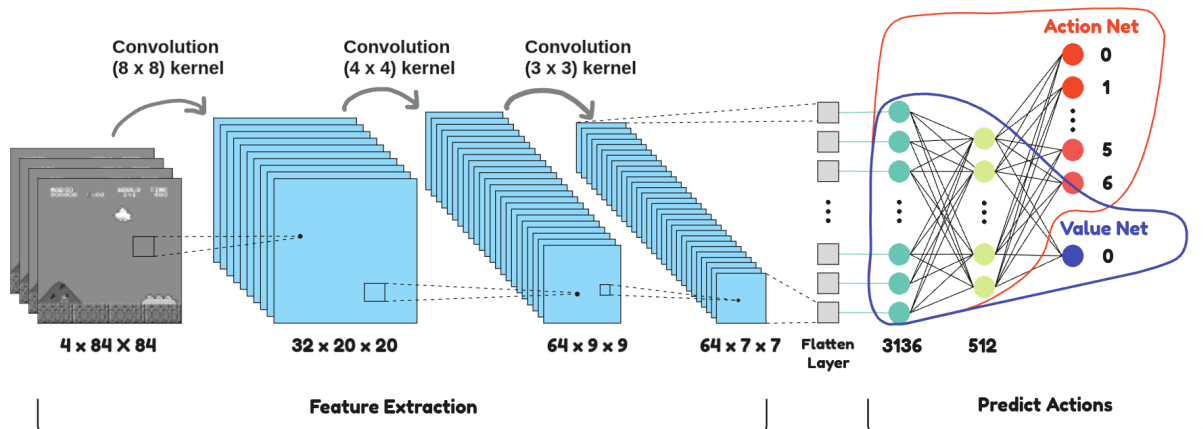


Figura 4.11: Arhitectura rețelei convoluționale pentru inputul de tip imagine

### 4.6.2 Hyperparametrii

Algoritmul PPO are următorii hyperparametri:

- **learning rate-ul** definește dimensiunea pasului de învățare din *backward pass*, cu alte cuvinte, cât de repede să-și modifice agentul greutatea și bias-urile din rețelele de actor și critic.
- **minibatch size-ul** definește numărul de pași care sunt colectați într-un buffer de rollout, care se împarte ulterior într-un număr de minibatch-uri ce sunt folosite în procesul de *backward propagation*.

- **minibatch size-ul** definește dimensiunea minibatch-ului care este selectat din bufferul de rollout. Un minibatch mic conduce către o alegere a datelor mai uniformă și mai aleatoare, împiedicând agentul din a da overfitting pe informațiile obținute din bufferul de rollout și de a se bloca într-o soluție suboptimă.
- **n epochs** este numărul de epoci pentru fiecare minibatch ales. De atâtea ori se va face *backward propagation* peste parametrii rețelei neurale din spatele modelul de PPO.
- **gamma** reprezintă factorul de discount din funcția de return care este folosit de către funcția obiectiv. De obicei acest hyperparametru ia valori foarte aproape de 1, cum ar fi 0.99, pentru a acorda importanță acțiunilor ce sunt luate mai târziu de către agent.
- **gae lambda** controlează diferența dintre varianța și bias-ul care sunt introduse în funcția de advantage. De obicei, această valoare este aproape de 1.
- **clip range-ul** determină cât de mult noua politică a rețelei se poate îndepărta de vechea politică. Un clip range mare ar da mai multă libertate agentului de a încorpora elemente din noua politică, dar agentul ar avea șanse mai mari de a face un pas greșit, rezultând în scăderea calității găsirii noilor politici, având astfel un efect negativ asupra întregului proces de învățare.
- **coeficientul de entropie** este un hyperparametru ce încurajează agentul să exploreze căi pe care nu le-a luat în trecut. Întrucât vrem ca agentul nostru să învețe să termine nivelul cât mai repede, vom seta această valoare cu 0.
- **coeficientul de value function** indică importanța pe care funcția de loss a *value function-ului* îl are în cadrul loss-ului total. Am lăsat această valoare la 0.5 pentru toți agenții antrenati.
- **max grad normalisation** este folosit pentru scalarea gradientilor și evitarea gradientilor *explozivi*. Am lăsat valoarea default de 0.5.

```
class stable_baselines3.ppo.PPO(policy, env, learning_rate=0.0003, n_steps=2048, batch_size=64,
n_epochs=10, gamma=0.99, gae_lambda=0.95, clip_range=0.2, clip_range_vf=None,
normalize_advantage=True, ent_coef=0.0, vf_coef=0.5, max_grad_norm=0.5, use_sde=False,
sde_sample_freq=-1, rollout_buffer_class=None, rollout_buffer_kwargs=None, target_kl=None,
stats_window_size=100, tensorboard_log=None, policy_kwargs=None, verbose=0, seed=None, device='auto',
_init_setup_model=True) [source]
```

Figura 4.12: Hyperparametrii default pentru PPO

## 4.7 Input de tip RAM

### 4.7.1 Memoria RAM

Pentru diversificarea rezultatelor, a fost folosit ca input memoria RAM a jocului. Jocul memorează un grid de 32 x 13 tile-uri, unde un tile ocupă 16 x 16 pixeli. Ecranul, care are dimensiunea de 16 x 13 tile-uri, doar scrolează prin acest grid coloană cu coloană iar atunci când ajunge la sfârșit, își dă wrap de la începutul grid-ului iar acesta se actualizează în memorie concomitent.

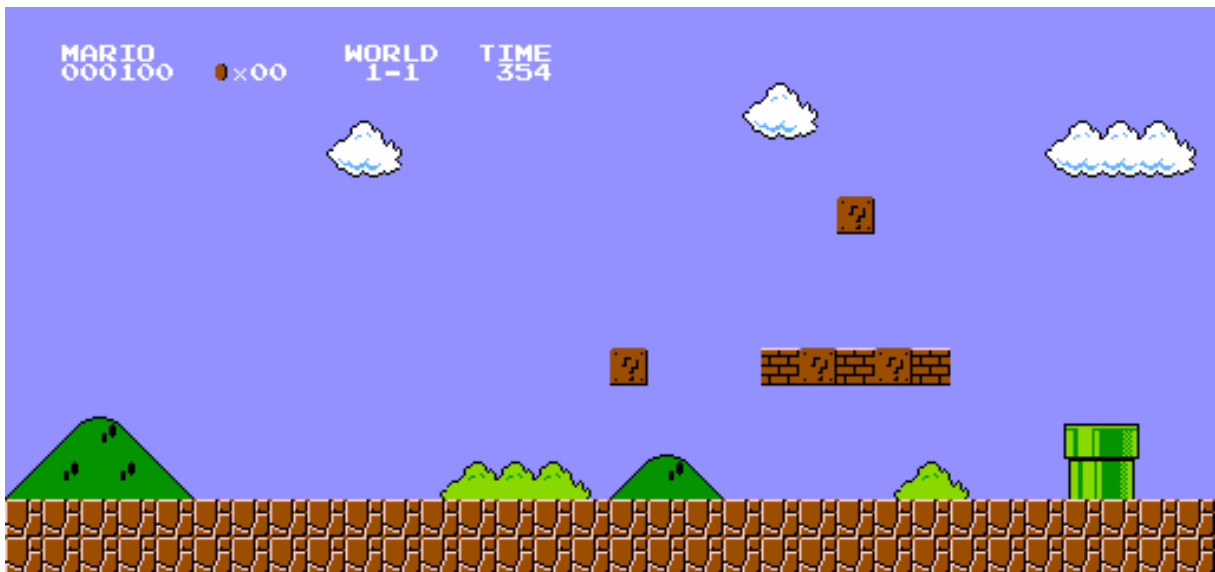


Figura 4.13: Tilemap-ul pentru Super Mario Bros

32

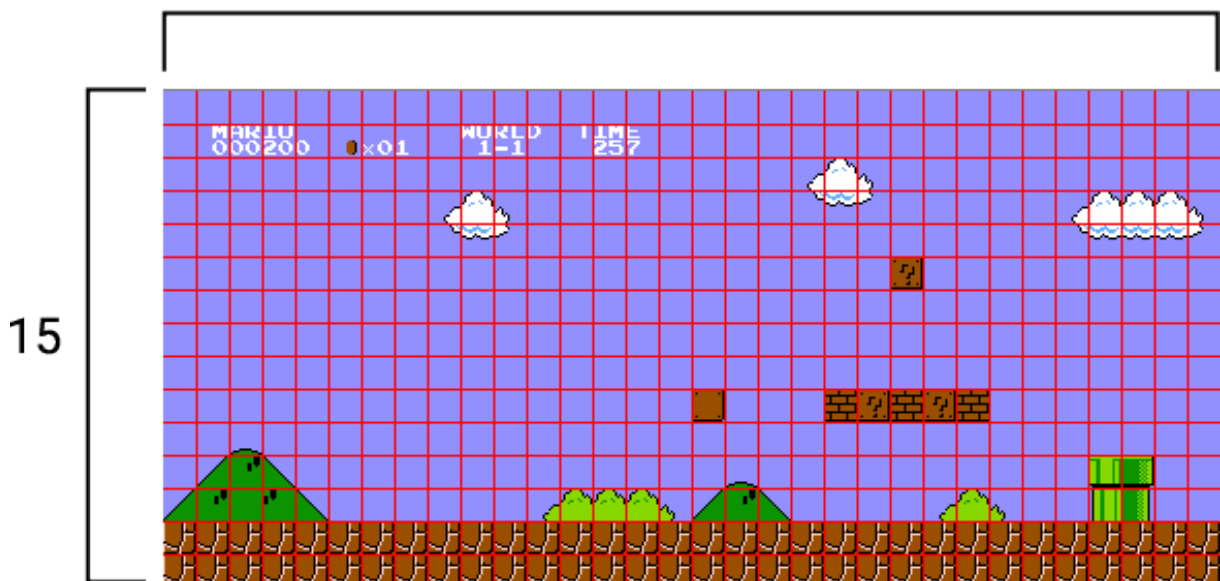


Figura 4.14: Grid-ul pentru Tilemap

### 4.7.2 Utils

A fost creat un script de Python numit **Super\_Mario\_RAM\_utils** unde au fost definite funcțiile de **get\_rendered\_screen** si **tile\_position\_to\_ram\_address**. Funcția de **tile\_position\_to\_ram\_address** returnează adresa din memoria RAM iar valoarea corespunzătoare acelei adrese a jocului este 0 în cazul în care avem doar fundalul pe acea pătrăciță din ecranul jocului și 1 in cazul in care avem un tile cu care agentul poate să interacționeze.

Funcția de *get\_rendered\_screen* returnează grid-ul final al ecranului pe care agentul o să-l obțină ca input. Funcția calculează matricea ecranului de 13 x 16 unde fiecare celulă care are 0 reprezintă fundalul, 1 daca este un tile, 2 daca reprezintă agentul, -1 pentru un inamic care poate fi *stomped* de către agent și -2 pentru inamici care sunt *unstompable* cum ar fi *Buzzy Beetle*.

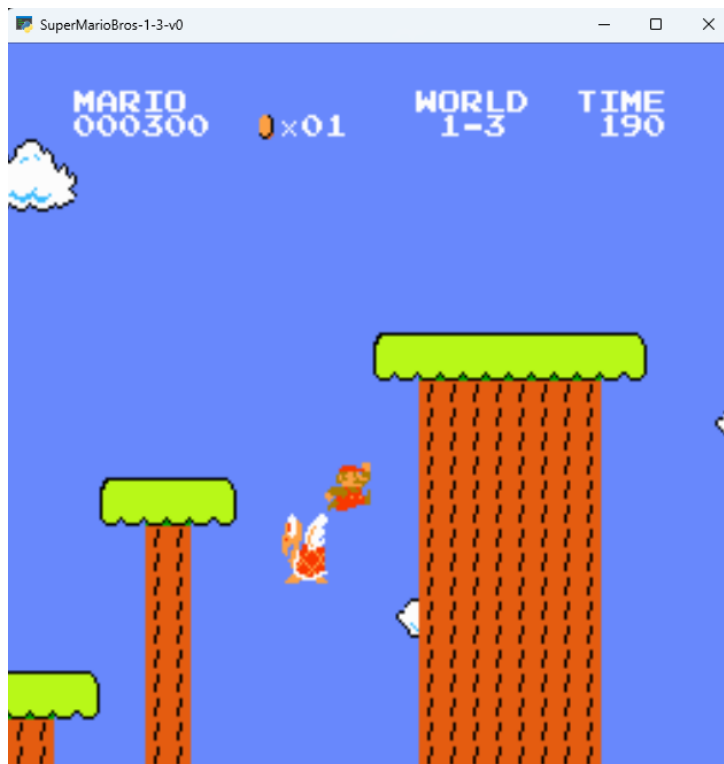


Figura 4.15: Un Koopa Troopa inainte de a fi *stompuit*



Figura 4.16: Un Blooper care nu poate fi *stompuit*

### 4.7.3 Frame Skipping si Frame Stacking

S-a creat un wrapper custom care dă skip la un număr de frame-uri înainte de a stivui fiecare frame în parte. Aceasta ajută pentru o mai bună vizualizare a mișcării agentului, scăpând de frame-urile redundante de care agentul nu are nevoie. Întrucât modelele primesc grid-ul ecranului de  $13 \times 16$  elemente ca input, fiecare element având 5 valori posibile, am redus dimensiunea inputului cu un factor uimitor de 34. De asemenea, dimensiunea spațiului de observații s-a restrâns de la  $(4 * 84 * 84)^{256}$  la  $(4 * 13 * 16)^5$ .

### 4.7.4 Arhitectura Rețelei

Pentru acest tip de input s-a folosit politica default de tip *MLP* (Multilayer Perceptron) din **stable-baselines3** pentru că nu mai este nevoie să fie extrase caracteristicile din cadrele de input. Politica de tip *MLP* se diferențiază de politica default de tip *CNN* prin lipsa de layere convoluționale, în locul lor aflându-se doar un layer de tip *Flatten* ce transformă tensorul de input într-un tensor cu 2 dimensiuni care reprezintă *batch size-ul* și dimensiunea totală a grid-ului.

Pentru secțiunea de *MLP extractor*, actorul și criticul au fiecare câte 2 layere liniare urmate fiecare de câte o funcție de activare de tip *Tanh*. Ultimul layer pentru actor este un layer liniar ce returnează probabilitățile celor 7 acțiuni disponibile agentului (după aplicarea algoritmului de *Softmax*), iar ultimul layer pentru critic este tot un layer liniar

care returnează estimarea funcției de value.

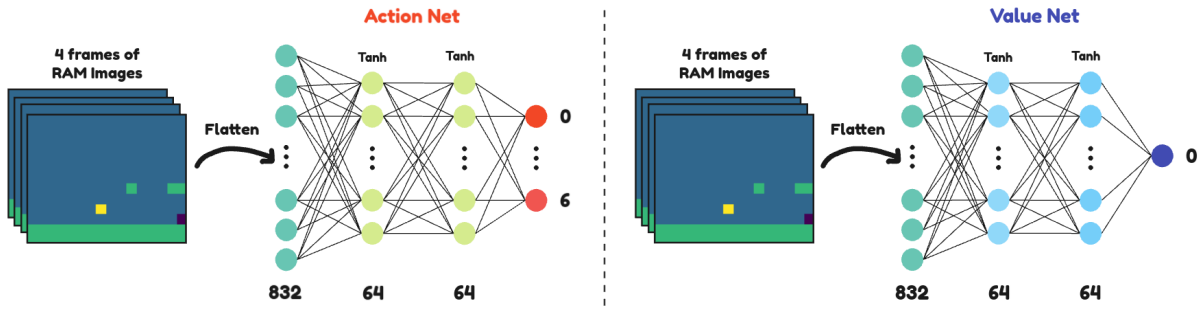


Figura 4.17: Arhitectura rețelei de tip MLP

### 4.7.5 Catastrophic Forgetting și Elastic Weight Consolidation

Agentul a reușit, după 5 milioane de pași, să învețe să joace primul nivel la perfecție pe modul determinist. O mare problemă ce apare în încercarea de a învăța modele preantrenate pe câteva cerințe predefinite să completeze alte cerințe este aceea de a motiva agentul de a nu se îndepărta de vechile politici în timp ce încearcă să găsească o politică propice noii cerințe. Această problemă poartă numele de *Catastrophic Forgetting*.

O soluție de a mitiga tendința modelului de a uita ce a învățat înainte este a de a adăuga un nou termen de *loss*, direct proporțional cu diferența dintre parametrii noii politici și cei ai vechii politici. Această soluție se numește Elastic Weight Consolidation sau EWC pe scurt. Ideea din spate constă în a acorda fiecărei greutate și bias din modelul preantrenat un termen care indică importanța pentru rezolvarea task-urilor anterioare. Cu cât noua politică încearcă să se îndepărteze de vechea politică prin modificarea acestor parametri, cu atât mai mult acești termeni vor penaliza modelul.

Pentru termenii folosiți pentru acordarea importanței fiecărui parametru din vechea rețea este folosită diagonala principală din matricea de *Fisher Information*  $F_i$ .  $\mathcal{L}_B(\theta)$  reprezintă funcția de loss pentru noul task,  $\theta_i$  reprezintă parametrii politicii curente iar  $\theta_{A,i}^*$  reprezintă parametrii politicii vechi.

$$\mathcal{L}(\theta) = \mathcal{L}_B(\theta) + \sum_i \frac{\lambda}{2} F_i (\theta_i - \theta_{A,i}^*)^2$$

Pe lângă hyperparametrii pentru algoritmul de PPO, mai adăugăm și hyperparametri pentru EWC, aceia fiind  $\lambda$

# Capitolul 5

## Evaluarea rezultatelor

### 5.1 Input de tip imagine

#### 5.1.1 Rata de învățare constantă

Learning rate-ul este unul dintre cei mai importanti hyperparametrii ai agentului întrucât el dictează cât de repede sau cât de încet un agent învață. Rata de învățare nu trebuie sa fie prea mică pentru că atunci agentul ar petrece mult timp încercând să învețe lucruri relativ simple dar nici prea mare întrucât aceasta ar putea destabiliza agentul, împiedicându-l din a converge către un maxim optim.

Au fost construiți doi agenți cu scopul de a fi comparate performanțele. S-a folosit o rată de învățare constantă și restul de hyperparametrii default.

În figura 5.1 se observă recompensele totale acumulate de către primul agent (culoarea neagră) care are learning rate-ul egal cu  $1e-4$  și cele obținute de către cel de-al doilea agent care are learning rate-ul egal cu  $1e-5$ . Al doilea agent învață mai eficient decât primul agent, deși comportamentul acestuia devine instabil după aproximativ 3 milioane de pași. De la acest punct încolo pasul de învățare devine prea mare și agentul începe să se îndepărteze de politica deja învățată, uitând astfel strategia învățată la începutul nivelului.

Același rezultat este evidențiat si de graficul din figura 5.2. Valoarea de *mean length* scade la începutul graficului, urmând ca aceasta să ramană constanta timp de aproximativ 2 milioane de pași. Spre sfârșitul perioadei de antrenare, metrica prezentată în grafic crește, semnificând că agentul uită tacticile învățate anterior.

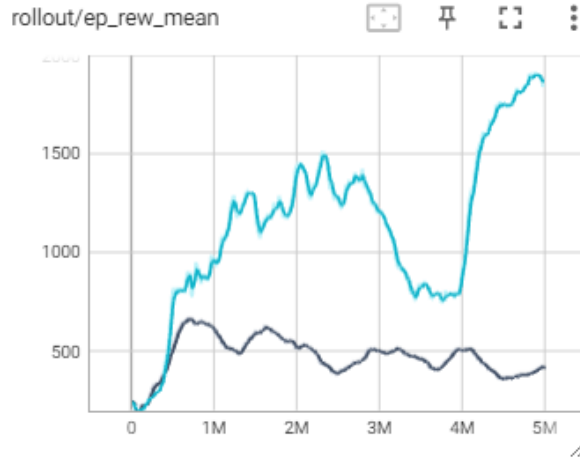


Figura 5.1: Mean rewardurile agenților cu rata de învățare constantă

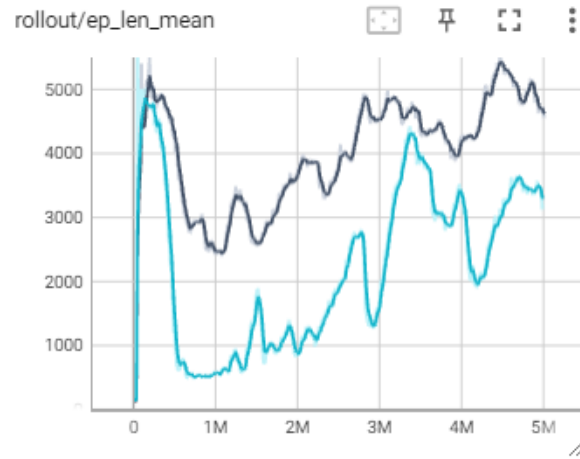


Figura 5.2: Mean lengthurile agentilor cu rata de învățare constantă

În figurile de mai jos, se observă graficele de *loss* și *explained variance*. Acestea oscilează foarte mult de-a lungul procesului de învățare, acest fapt fiind un indicator al ratei de învățare mult prea mare dat agentului.



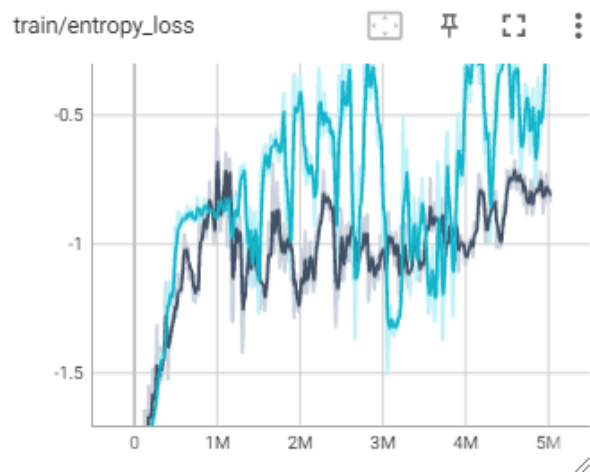


Figura 5.3: Metrica de entropy loss ale agenților cu rata de învățare constantă

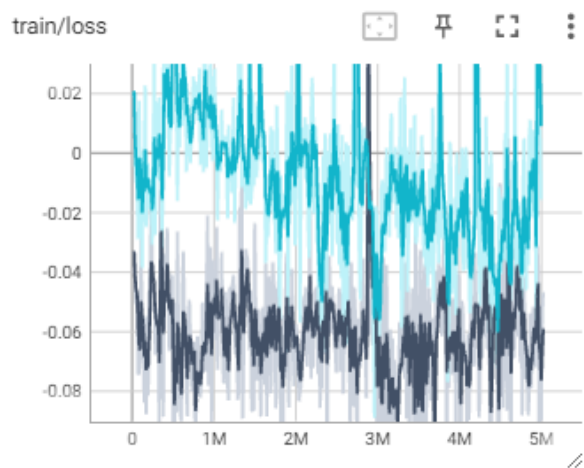


Figura 5.4: Metrica de train loss ale agenților cu rata de învățare constantă

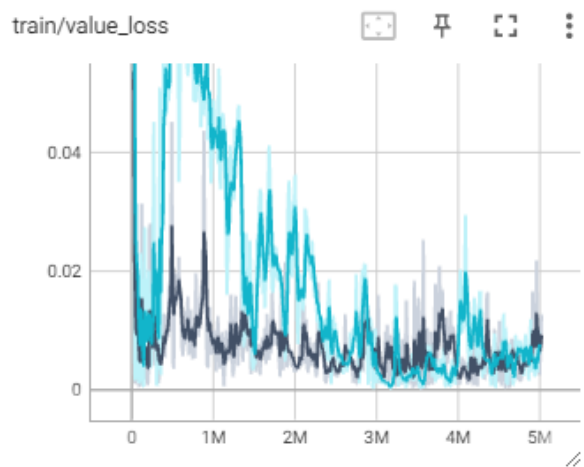


Figura 5.5: Metrica de value loss ale agenților cu rata de învățare constantă

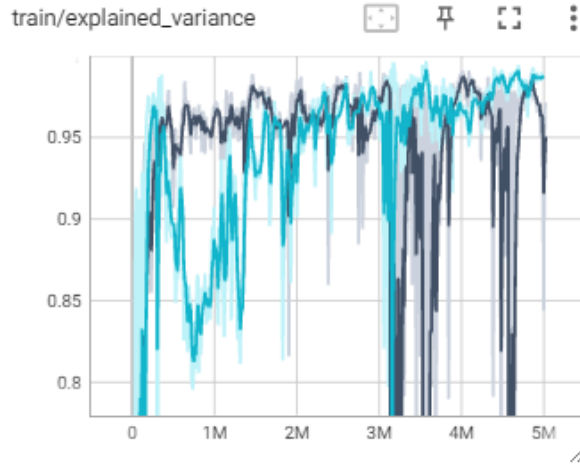


Figura 5.6: Metrica de explained variance ale agenților cu rata de învățare constantă

### 5.1.2 Rata de învățare liniară

Un mod diferit de a experimenta cu input de tip imagine este aplicarea unei rate de învățare liniară. Pentru acest tip de rată de învățare, agentul controlează mărimea pasului de învățare prin intermediul unei funcții ce returnează produsul dintre progresul rămas, care este reprezentat ca un număr subunitar, și valoarea inițială pentru *learning rate*.

```
def linear_schedule(initial_value: float):
    def func(progress_remaining: float) -> float:
        return progress_remaining * initial_value

    return func
```

Listing 5.1: Linear Learning Rate Scheduler

S-au testat în total 9 modele cu rata de învățare liniară, fiind modificate numai hyperparametrii de *batch size*, *minibatch size*, *learning rate* și *n epochs*. Ceilalți hyperparametrii, cum ar fi *gamma* sau *clip range*, au fost păstrate ca fiind cele default din algoritmul de *PPO* din pachetul de **stable-baselines3**.

În urma rezultatelor obținute de pe urma antrenării acestor agenți, s-a observat că modelul de *Super Mario Bros* care primește ca input ultimele 4 frame-uri ale jocului sub forma unei matrici de dimensiunea  $(4, 84, 84)$  are nevoie de un buffer de rollout de cel puțin 2048 de pași și un *minibatch size* direct proporțional cu rata de învățare. Cu toate acestea, un *minibatch size* mic precum 64 ajută agentul să învețe mai uniform peste toate episoadele colectate în buffer-ul de rollout. Primele trei modele au fost cele care au obținut un *mean reward* satisfăcător.

Nici un agent care a primit ca input ultimele 4 cadre ale jocului nu a reușit să termine nivelul pe modul determinist. Însă, pe modul stocastic, primele 3 modele reușesc să termine primul nivel. Acest rezultat se poate datora și arhitecturii rețelei neurale convo-

| Learning Rate | Batch Size | Minibatch Size | Number of epochs | Mean Reward |
|---------------|------------|----------------|------------------|-------------|
| 5e-5          | 2048       | 32             | 2                | 1643        |
| 1e-4          | 4096       | 256            | 2                | 1464        |
| 5e-4          | 4096       | 256            | 5                | 1434        |
| 1e-4          | 4096       | 512            | 5                | 354         |
| 5e-4          | 4096       | 512            | 5                | 317         |
| 1e-4          | 4096       | 512            | 2                | 340         |
| 5e-4          | 4096       | 64             | 5                | 247         |
| 1e-4          | 2048       | 64             | 5                | 300         |
| 1e-4          | 512        | 64             | 10               | 218         |

Tabela 5.1: Tabela cu agentii antrenati folosind linear learning.

luționale, a cărui complexitate a fost prea mică pentru cerința dată. Din cauza aceasta, agentul a fost încurajat să dea *overfitting* în loc să generalizeze mecanicile jucătorului.

## 5.2 Input de tip RAM

S-a experimentat cu rețeaua default de tip *MLP* utilizându-se o rată de învățare liniară de  $3e-4$ . A fost ales un *linear learning rate* mai mare din cauză că, din moment ce modelul este unul de tip *MLP*, agentului îi ia mai puțin timp și poate să folosească un *learning step* mai mare pentru a învăța politica corectă. Așa cum se poate vedea din figurile de mai jos, antrenamentul agentului este unul stabil. *Mean length-ul* scade repede iar *mean reward-ul* crește constant. Graficul de *train loss* din figura 5.11 scade constant timp de aproximativ 3 milioane de pași, urmând să penduleze pentru următoarele 2 milioane de operații. Aceasta poate însemna că agentul, după ce și-a construit mare parte din politica optimă, încearcă să învețe micile detalii peste care trebuie să se perfecționeze, cum ar fi săritul peste inamicii de la mijlocul nivelului și peste gropile care îi sunt puse în cale.

De asemenea, graficele de *approximate KL* și *explained variance* din figurile 5.10 și 5.9 cresc pe tot parcursul procesului de învățare, acest lucru indicând că politicile noi se diferențiază de cele vechi, criticul îmbunătățindu-și abilitatea de a prezice mișcările actorului.

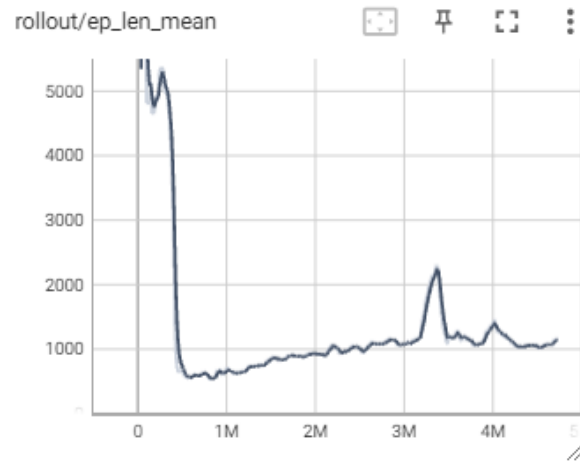


Figura 5.7: Mean length-urile agentului

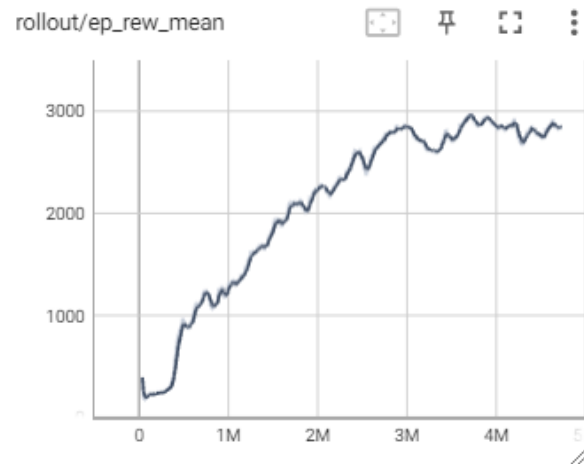


Figura 5.8: Mean reward-urile agentului

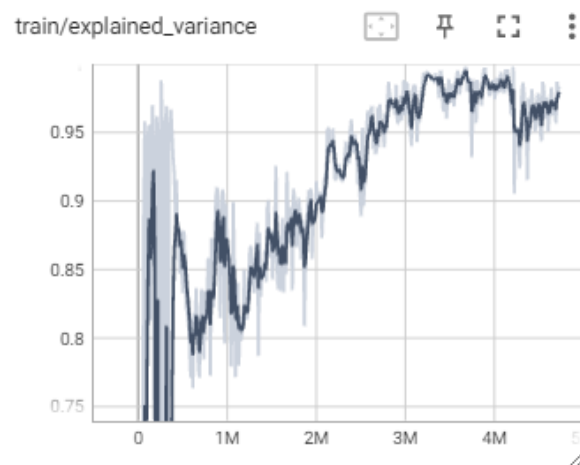


Figura 5.9: Metrica de explained variance al agentului

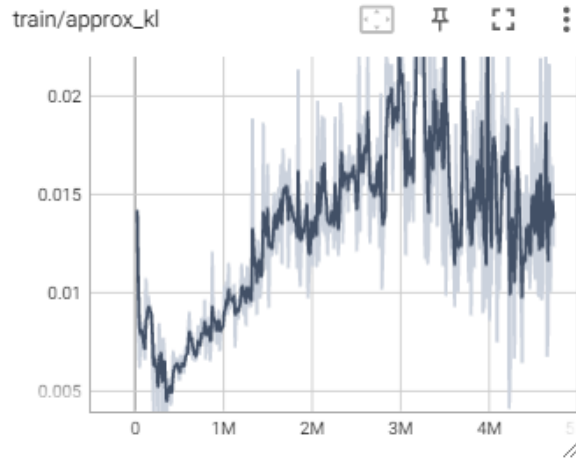


Figura 5.10: Metrica de KL divergence al agentului

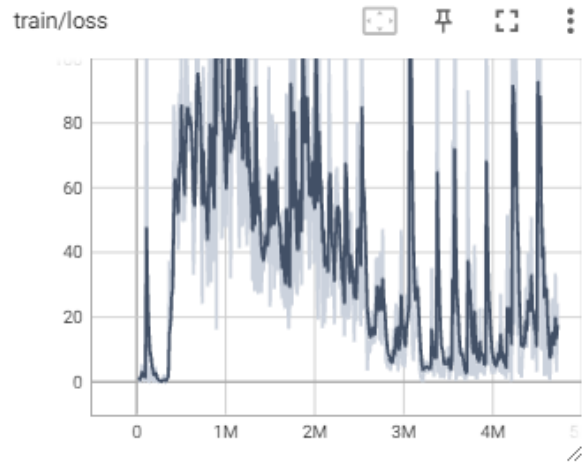


Figura 5.11: Metrica de train loss al agentului

### 5.3 Elastic Weight Consolidation

S-a experimentat cu 3 combinații diferite de  $\lambda$  și *initial learning rate*, nemodificând ceilalți hyperparametri de la modelul preantrenat.

| Lambda | Learning Rate | Mean Reward |
|--------|---------------|-------------|
| 1      | 1e-4          | 1438        |
| 0.8    | 1e-4          | 1610        |
| 1      | 5e-5          | 782         |

Tabela 5.2: Tabela cu agenții antrenați folosind linear learnings și EWC

Așa cum se poate vedea din graficele din figura 5.7, primii doi agenți reușesc să obțină un *mean reward* rezonabil, ajungând până la mijlocul nivelului al doilea. Cu ajutorul

termenului de loss calculat folosind metoda de *EWC*, agenții nu uită să joace pe primul nivel, cu excepția ultimei părți, unde în loc să sară peste țeava verde sau platformele maro, agenții stau pe loc. Aceasta poate fi o urmare a faptului că  $\lambda$ -ul este prea mic. O valoare de  $\lambda$  mai mare ar putea ajuta agentul să-și fixeze mai strâns parametrii din rețeaua neurală pentru a nu uita cum să completeze primul nivel.

De asemenea, se poate observa că pentru agentul cu  $\lambda$  0.8 și același learning rate ca primul agent, metrica de *mean reward* crește mai repede decât la ceilalți agenți, datorită importanței mai mici pe care modelul le atribuie greutăților agentului preantrenat. Ultimul agent nu reușește să convergă la fel de bine ca ceilalți agenți datorită learning rate-ului inițial de  $5e-5$ , ceea ce semnifică faptul că *learning step-ul* este prea mic pentru update-ul politicii.

Deși procesul de antrenare durează mai mult decât pentru primul nivel din cauza adăugării unui termen de regularizare ce încurajează agenții să stea în apropierea politicii precaculate, rezultatele obținute sunt satisfăcătoare.

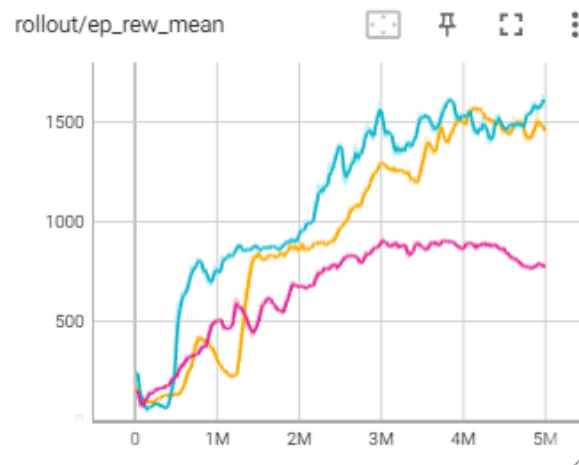


Figura 5.12: Lungimea medie a unui episod pentru modelul de EWC

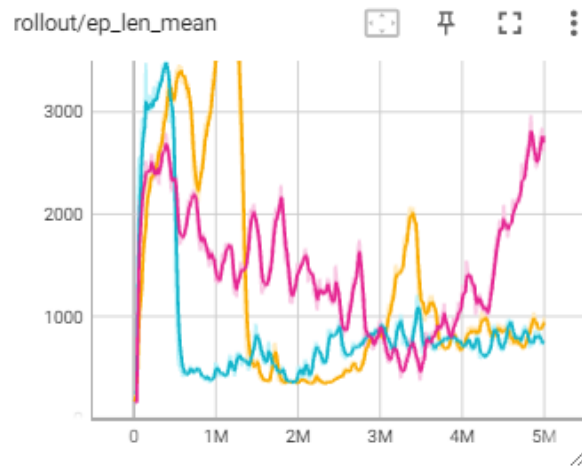


Figura 5.13: Recompensa totală medie a unui episod pentru modelul de EWC



Figura 5.14: Agentul se blochează la sfârșitul primului nivel după ce se antrenează pe nivelul 2

# Capitolul 6

## Concluzie

În urma antrenării și analizării unui număr de 15 agenți cu un total de aproximativ 75 de milioane de pași s-a ajuns la concluzia ca cele mai bune rezultate s-au obținut în cadrul algoritmului de PPO prin folosirea inputului de tip RAM, urmate de adăugarea metodei de Elastic Weight Consolidation pentru prevenirea problemei de *Catastrophic Forgetting*.

Abordarea prezentată în această lucrare privind dezvoltarea, testarea și îmbunătățirea agentului de *Deep Reinforcement Learning* pentru completarea primelor două nivele din Super Mario Bros prezintă numeroase aspecte ce pot fi în continuare cercetate și consolidate:

- **Complexitatea arhitecturii:** Toți agenții cu care am experimentat pentru inputul de tip RAM au avut aceeași arhitectură formată din câte două layere ascunse de câte 64 de neuroni. O posibilă cauză a uitării pe care agenții ce folosesc *EWC* o prezintă este prezența de *overfitting* după antrenarea pe primul nivel. Astfel, atunci când același agent a fost antrenat și pentru nivelul 2, modelul a întâmpinat greutăți în a-și modifica politica curentă din cauza lipsei de complexitate a arhitecturii neurale.
- **Buget mărit:** Pentru antrenarea agenților cu *EWC* pentru al doilea nivel, am folosit un buget de 5 milioane de operații, nelăsând agenții să converge către un maximum optim. Din cauza aceasta, *learning rate-ul* scade la o rată rapidă până când pașii de antrenare devin prea mici pentru a lăsa noua politică de a se diferenția semnificativ de cea veche.
- **On-policy vs Off-policy:** Deși algoritmi de on-policy, în special algoritmul de *Proximal Policy Optimization*, sunt mult mai stabili decât cei de off-policy sau cei care se bazează pe un model al mediului, merită încercat antrenamentul agenților și cu alți algoritmi de *Deep Reinforcement Learning*. În special, algoritmi de off-policy pot reprezenta o alternativă mai bună decât algoritmul de *PPO* din cauza eficienței alegerii de sample-uri pentru calcularea următoarei politici. Astfel, *mean*



*reward-ul* poate crește semnificativ mai repede și agenții pot să convergă către un maximum optim fără să dea semne de *overfitting*.

- **Implementarea manuală:** Pentru codarea acestui proiect s-a folosit framework-ul de **stable-baselines3** care prezintă multe wrappere și logică deja implementate ce pot fi folosite direct de către programator pentru administrarea rapidă a agenților de *DRL*. Deși folosirea unei asemenea librării are avantajele ei, există multe dezavantaje ce pot trage în jos perspectiva folosirii acesteia în viitor. Unul dintre cele mai mari inconveniențe este lipsa de modularitate al framework-ului, fapt care a obligat, în prezentul experiment, modificarea codului sursă al algoritmului de *PPO* și al altor funcții utilitare pentru a încorpora metoda de *Elastic Weight Consolidation*.

# Bibliografie

- [1] DeepMind, *AlphaGo*, <https://deepmind.google/technologies/alphago/>, Accesat: 2024-06-08.
- [2] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska et al., „Overcoming catastrophic forgetting in neural networks”, în *Proceedings of the national academy of sciences* 114.13 (2017), pp. 3521–3526.
- [3] David Gregory LeBlanc și Greg Lee, „General Deep Reinforcement Learning in NES Games.”, în *Canadian Conference on AI*, 2021.
- [4] OpenAI, *OpenAI Five defeats Dota 2 world champions*, (accesat la data de 3 iunie 2024), 2019, URL: <https://openai.com/index/openai-five-defeats-dota-2-world-champions/>.
- [5] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus și Noah Dormann, „Stable-baselines3: Reliable reinforcement learning implementations”, în *Journal of Machine Learning Research* 22.268 (2021), pp. 1–8.
- [6] John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan și Philipp Moritz, „Trust region policy optimization”, în *International conference on machine learning*, PMLR, 2015, pp. 1889–1897.
- [7] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford și Oleg Klimov, „Proximal policy optimization algorithms”, în *arXiv preprint arXiv:1707.06347* (2017).
- [8] Yumou Wei, *Super Mario Bros Reinforcement Learning*, (accesat la data de 3 iunie 2024), 2022, URL: <https://github.com/yumouwei/super-mario-bros-reinforcement-learning>.